ICOM 3° · PROGRAMACIÓN ORIENTADA A OBJETOS

Java desde cero

Guía de estudio para tu examen de POO

Explicada paso a paso, con ejemplos sacados de tus propios ejercicios de clase: los futbolistas, el hotel, las figuras geométricas y la lista de usuarios.

```java
public class Estudiante {
    private String meta = "Pasar el examen de POO";

    public void estudiar(int dias) {
        for (int dia = 1; dia <= dias; dia++) {
            System.out.println("Día " + dia + ": una lección más");
        }
    }
}
```

TU RUTA DE ESTUDIO

1. Tu primer programa
2. Decisiones y ciclos
3. Clases y objetos
4. Encapsula-miento y static
5. Herencia
6. Abstracción y polimorfismo
7. Arreglos y excepciones

7
lecciones, una por día

32
prácticas con sus soluciones

1
examen de práctica completo

Todos los programas de esta guía fueron compilados y ejecutados.

Octubre 2026

Contenido

Cómo usar esta guía 4

Lección 1 · Tu primer programa 6

1.1 ¿Qué es Java y cómo funciona? 6
1.2 El esqueleto de todo programa 7
1.3 Variables: cajitas con nombre 8
1.4 Operadores y la trampa de la división 9
1.5 Mostrar mensajes en pantalla 9
1.6 Leer del teclado con Scanner 10
1.7 Comparar textos: equals, no == 12
Prácticas de la Lección 1 13

Lección 2 · Decisiones y ciclos 15

2.1 Comparaciones y condiciones 15
2.2 if, else if y else 15
2.3 switch: elegir entre varias opciones 16
2.4 El ciclo for 17
2.5 while y do-while 18
2.6 El patrón del menú 19
2.7 Pseudocódigo y diagramas de flujo 20
Prácticas de la Lección 2 21

Lección 3 · Clases y objetos 23

3.1 ¿Qué es la Programación Orientada a Objetos? 23
3.2 Las partes de una clase 24
3.3 Crear y usar objetos 25
3.4 El constructor 25
3.5 La palabra this 26
3.6 Métodos: acciones que reciben y regresan datos 27
3.7 El diagrama de clases (UML) 28
Prácticas de la Lección 3 29

Lección 4 · Encapsulamiento y static 32

4.1 El problema: datos sin protección 32
4.2 Modificadores de acceso 33
4.3 Getters y setters 33
4.4 static: lo que pertenece a la clase 35
Prácticas de la Lección 4 36

Lección 5 · Herencia 39

5.1 La idea: «es un» 39
5.2 extends: declarar la herencia 39
5.3 super: llamar al constructor del padre 40
5.4 El orden de los constructores 41
5.5 Sobrescribir métodos y reusar los del padre 41
5.6 La herencia en el diagrama de clases 42
Prácticas de la Lección 5 43

Lección 6 · Abstracción y polimorfismo 45

6.1 Sobrescritura (@Override) 45

6.2 Sobrecarga 45
6.3 Abstracción y clases abstractas 46
6.4 Polimorfismo: un mensaje, muchas formas 48
6.5 Los 4 pilares de la POO 50
Prácticas de la Lección 6 50

Lección 7 · Arreglos, ArrayList y excepciones 53
7.1 Arreglos: muchas cajitas en fila 53
7.2 Arreglos de objetos 54
7.3 ArrayList: la lista que crece sola 55
7.4 Imprimir objetos: toString() 56
7.5 Excepciones: cuando algo sale mal 57
Prácticas de la Lección 7 59

Anexo · Diagramas de casos de uso 61

Chuleta de repaso 63

Examen de práctica 65
I. Teoría 65
II. ¿Qué imprime? 66
III. Encuentra y corrige los errores 68
IV. Del diagrama al código 69
V. Diagrama de casos de uso 69

Soluciones 70

Cómo usar esta guía

Esta guía parte **desde cero**. No necesitas saber nada de Java para empezar: cada lección explica las ideas con calma, usa ejemplos de **tus propios ejercicios de clase** y termina con prácticas para que compruebes que de verdad lo entendiste.

La meta no es memorizar código, sino que puedas **leer un programa y explicar qué hace cada línea**. Cuando logras eso, escribir tu propio código sale mucho más fácil.

Plan de estudio: una lección por día

Día	Lección	Temas
1	**Tu primer programa**	Estructura de un programa, variables, operadores, imprimir y leer con `Scanner`
2	**Decisiones y ciclos**	`if`, `switch`, `for`, `while`, `do-while`, menús, pseudocódigo
3	**Clases y objetos**	Atributos, métodos, constructor, `this`, `new`, diagrama de clases
4	**Encapsulamiento y static**	`private` / `public` / `protected`, getters y setters, `static`
5	**Herencia**	`extends`, `super`, orden de constructores, sobrescribir
6	**Abstracción y polimorfismo**	Clases abstractas, sobrecarga vs. sobrescritura, los 4 pilares
7	**Arreglos, ArrayList y excepciones**	Arreglos de objetos, listas, `try` / `catch`. Después: el examen de práctica

¿Tienes menos días? Junta las lecciones 1 y 2 (son lo básico de programación) y date más tiempo para las lecciones 3 a 6, que son el corazón de la materia.

Cómo estudiar cada lección

1. **Lee la lección completa una vez**, sin prisa y sin saltarte los recuadros.
2. **Copia a mano los ejemplos de código.** Sí, a mano. Escribir te obliga a fijarte en cada `;`, cada `{` y cada mayúscula. Y si tu examen es en papel, es justo lo que vas a hacer ahí.
3. **Resuelve las prácticas sin ver las respuestas.** Equivocarte aquí es la mejor forma de no equivocarte en el examen.
4. **Revisa con las soluciones** del final de la guía y corrige con pluma de otro color.
5. **Si tienes la compu a la mano, escribe el programa y ejecútalo** para comprobar que hace lo que pensabas.
6. **Lo que no entiendas, márcalo con un «?»** y pregúntamelo en el chat (o a tu maestra).

Los recuadros de esta guía

> 💡 **IDEA CLAVE**
> Lo más importante de la sección. Si solo recuerdas una cosa, que sea esta.

> ⚠️ **CUIDADO**
> Errores muy comunes y trampas que hacen fallar un programa.

🎯 **PARA EL EXAMEN**
Definiciones y preguntas que suelen aparecer en los exámenes.

🧩 **ANALOGÍA**
Una comparación con la vida diaria para entender la idea.

✅ **TIP**
Trucos y atajos que te facilitan la vida.

🔎 **DETECTIVE**
Un error real escondido en tus propios ejercicios.

En los bloques de código, las líneas con fondo amarillo son las importantes. Los bloques naranjas muestran lo que aparece en la consola; `en verde`, lo que escribe el usuario.

Cómo ejecutar los ejemplos

En NetBeans (como en clase)

Crea un proyecto (*File → New Project → Java with Maven → Java Application*), escribe el código y presiona **F6** (*Run*). Si el ejemplo tiene varias clases públicas, crea un archivo por clase: clic derecho en el paquete → *New → Java Class*.

En la terminal

Guarda el código en un archivo que se llame igual que la clase y ejecuta:

Terminal

```
javac Ejemplo.java
java Ejemplo
```

Si son varias clases en una carpeta: `javac *.java` y luego `java` + la clase que tiene el `main`.

Algunos ejemplos traen varias clases en un mismo bloque para ahorrar espacio. Se pueden guardar juntas en un solo archivo, siempre que solo una sea `public` (la que tiene el `main`) y el archivo lleve su nombre.

1

LECCIÓN 1 · DÍA 1

Tu primer programa

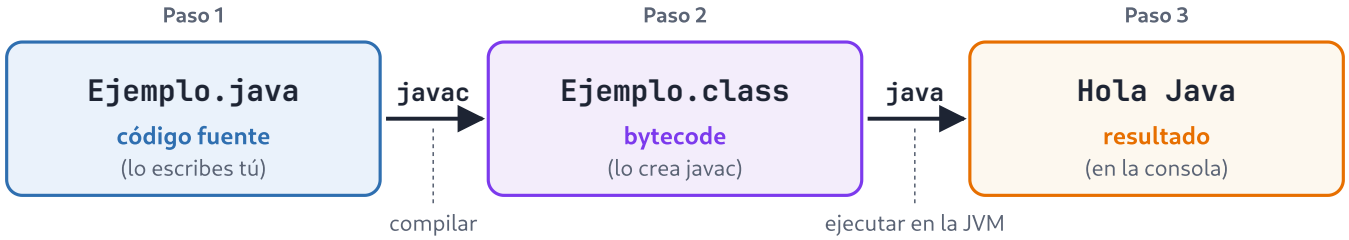
Cómo se ve un programa en Java, variables, operaciones, y cómo mostrar y leer datos

Vas a aprender:

- Qué pasa desde que escribes código hasta que corre
- Las partes obligatorias de todo programa
- Guardar datos en variables
- Hacer operaciones (y la trampa de la división)
- Mostrar mensajes en pantalla
- Leer lo que escribe el usuario con `Scanner`

1.1 ¿Qué es Java y cómo funciona?

Java es un **lenguaje de programación de alto nivel**: escribes instrucciones con palabras parecidas al inglés (`class`, `if`, `while`) en lugar de ceros y unos. Para que la computadora lo entienda, tu programa pasa por tres pasos:



1. **Escribes** el código en un archivo que termina en `.java`. A eso se le llama **código fuente**.
2. **Compilas** con `javac`: el compilador revisa que no haya errores de escritura y traduce tu código a **bytecode** (un archivo `.class`). Si hay un error (te faltó un `;`, por ejemplo), aquí te avisa y no sigue.
3. **Ejecutas** con `java`: la **JVM** (Máquina Virtual de Java) lee el bytecode y lo pone a funcionar.

En NetBeans los pasos 2 y 3 ocurren juntos cuando presionas **Run** (F6).

🎯 PARA EL EXAMEN: CARACTERÍSTICAS DE JAVA

En tu nota del 21 de agosto esta lista quedó incompleta. Estas son las que más se preguntan:

- **Orientado a objetos:** los programas se organizan en clases y objetos.
- **Multiplataforma (portable):** el mismo `.class` corre en Windows, Mac o Linux, porque cada sistema tiene su propia JVM. «Escríbelo una vez, ejecútalo donde sea».
- **Fuertemente tipado:** cada variable tiene un tipo fijo (`int`, `String`…) que no puede cambiar.
- **Robusto y seguro:** el compilador detecta muchos errores antes de ejecutar, y la memoria se libera sola (el *recolector de basura*).
- **De alto nivel:** se parece más al lenguaje humano que al de la máquina.

JDK (*Java Development Kit*): el kit con todo lo necesario para **crear** programas: el compilador `javac`, el comando `java` y las bibliotecas. **JVM**: la máquina virtual que **ejecuta** los programas.

1.2 El esqueleto de todo programa

Este fue tu primer programa en clase. Todo programa en Java tiene, como mínimo, esta forma:

Ejemplo.java

```java
public class Ejemplo {                          // ① la clase
    public static void main(String[] args) {    // ② el método main
        System.out.println("Hola Java");        // ③ una instrucción
    }                                           // ④ cierra main
}                                               //   cierra la clase
```

Consola

```
Hola Java
```

1. `public class Ejemplo`: en Java **todo el código vive dentro de una clase**. Regla de oro: el archivo debe llamarse **exactamente** igual que la clase pública, con todo y mayúsculas: `Ejemplo.java`.
2. `public static void main(String[] args)`: el método `main` es la **puerta de entrada**. Java empieza a ejecutar aquí, línea por línea, de arriba hacia abajo. Por ahora apréndetelo de memoria; en la Lección 4 entenderás qué significa `static`. (En tu primer ejercicio aparece como `String [ ] arg`: es lo mismo, solo cambian los espacios y el nombre.)
3. `System.out.println("Hola Java");` es una **instrucción**. Toda instrucción termina con **punto y coma** `;`.
4. **Las llaves** `{ }` abren y cierran bloques: la clase tiene las suyas y `main` tiene las suyas, una dentro de la otra. Cada `{` necesita su `}`.

> ⚠️ **CUIDADO: JAVA DISTINGUE MAYÚSCULAS DE MINÚSCULAS**
>
> `System` no es lo mismo que `system`, ni `String` que `string`, ni `main` que `Main`. Los errores más comunes al empezar son:
>
> - Olvidar el `;` al final de una instrucción.
> - Abrir una llave `{` y no cerrarla.
> - Escribir mal una mayúscula (`system.out.println`).
> - Que el archivo no se llame igual que la clase.

Comentarios

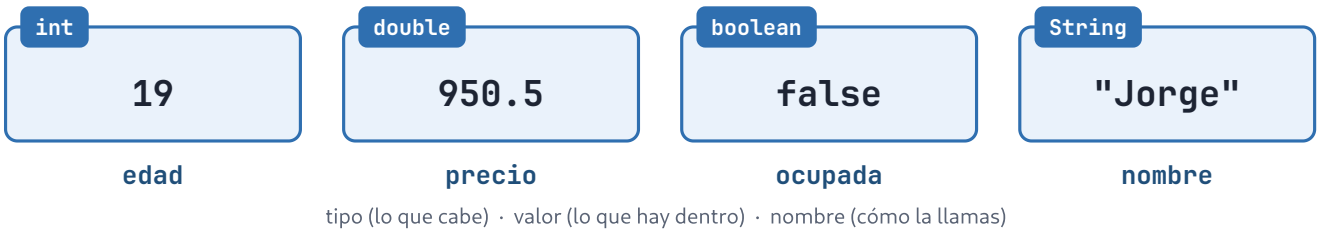
Los comentarios son notas para las personas: el compilador los ignora. Sirven para explicar tu código (tus proyectos están llenos de ellos, ¡y está muy bien!).

```java
// Comentario de una sola línea

/* Comentario de
   varias líneas */
```

1.3 Variables: cajitas con nombre

Una **variable** es como una cajita con nombre donde guardas un dato. En Java, además del nombre, cada cajita tiene un **tipo** que dice qué clase de dato cabe en ella.



tipo (lo que cabe) · valor (lo que hay dentro) · nombre (cómo la llamas)

Se declaran así: `tipo nombre = valor;`

```
int edad = 19;              // declarar y dar valor de una vez
double precio;              // declarar sin valor...
precio = 950.5;             // ...y asignarlo después
edad = 20;                  // cambiar el valor (ya NO se repite el tipo)
```

Tipo	Qué guarda	Ejemplo
`int`	Números enteros (sin punto decimal)	`int dorsal = 10;`
`double`	Números con decimales	`double precioPorNoche = 800.0;`
`boolean`	Solo `true` (verdadero) o `false` (falso)	`boolean ocupada = false;`
`char`	Un solo carácter, entre comillas **simples**	`char opcion = 's';`
`String`	Texto, entre comillas **dobles**	`String nombre = "Jorge";`

> 💡 **IDEA CLAVE: STRING VA CON MAYÚSCULA**
>
> `int`, `double`, `boolean` y `char` son **tipos primitivos** (datos simples). `String` es una **clase**: por eso empieza con mayúscula, por eso tiene métodos como `.equals()` o `.length()`, y por eso se compara diferente (lo verás en la sección 1.7).

Reglas para poner nombres

- Empiezan con letra, nunca con número: `jugador1` ✔ `1jugador` ✘
- Sin espacios: `precioPorNoche` ✔ `precio por noche` ✘
- No pueden ser palabras reservadas de Java como `class`, `int` o `public` ✘
- Convención **camelCase**: la primera palabra en minúscula y cada palabra nueva con mayúscula: `numeroJugadores`, `contadorJugadores`.
- Las **clases** empiezan con Mayúscula (`Persona`, `Futbolista`); las **variables y métodos**, con minúscula (`nombre`, `getEdad()`).

1.4 Operadores y la trampa de la división

```java
int a = 7;
int b = 2;
System.out.println(a + b);    // suma            → 9
System.out.println(a - b);    // resta           → 5
System.out.println(a * b);    // multiplicación  → 14
System.out.println(a / b);    // división entera → 3   (¡no 3.5!)
System.out.println(a % b);    // residuo         → 1   (7 = 2·3 + 1)
System.out.println(7.0 / 2);  // con decimales   → 3.5
```

> ⚠️ **CUIDADO: LA DIVISIÓN ENTERA**
>
> Si los dos números son `int`, el resultado también es `int`: Java **corta** los decimales (no redondea). `7 / 2` da `3`, y `9 / 10` da `0`. Si al menos uno de los dos es `double`, el resultado conserva los decimales: `7.0 / 2` da `3.5`.

El operador `%` (módulo) da el **residuo** de la división. Su uso más famoso: saber si un número es par, porque un número par dividido entre 2 deja residuo 0: `numero % 2 == 0`.

Atajos que verás por todos lados:

Atajo	Significa	Ejemplo
`x++`	`x = x + 1`	`contadorJugadores++;` (de tu proyecto de futbolistas)
`x--`	`x = x - 1`	`vidas--;`
`x += 5`	`x = x + 5`	`total += precio;`

1.5 Mostrar mensajes en pantalla

`System.out.println(...)` imprime y **salta de línea** al final. `System.out.print(...)` imprime y se queda en la misma línea (así dibujabas los asteriscos de tus figuras). Con `+` pegas (**concatenas**) texto con variables:

```java
String nombre = "Ana";
int edad = 19;
System.out.println("Hola " + nombre + ", tienes " + edad + " años");
System.out.print("Sin salto... ");
System.out.println("¡pegado!");
System.out.println("Línea 1\nLínea 2");      // \n = salto de línea dentro del texto
```

Consola

```
Hola Ana, tienes 19 años
Sin salto... ¡pegado!
Línea 1
Línea 2
```

La trampa del +

Java lee el `+` de **izquierda a derecha**. Mientras solo haya números, suma; en cuanto aparece un texto, todo lo que sigue se **pega** como texto:

```
System.out.println("Edad: " + 18 + 1);    // pega "18" y luego pega "1"
System.out.println("Edad: " + (18 + 1));  // el paréntesis suma primero
System.out.println(18 + 1 + " años");     // suma 18+1 y luego pega el texto
```

Consola

```
Edad: 181
Edad: 19
19 años
```

> ✅ **TIP**
> Si quieres hacer una operación dentro de un mensaje, ponla **entre paréntesis**: `"Total: " + (precio * noches)` .

1.6 Leer del teclado con Scanner

Para que el usuario escriba datos se usa la clase `Scanner`. Son tres pasos:

1. **Importarla** hasta arriba del archivo: `import java.util.Scanner;` (o `import java.util.*;`, que importa todo el paquete, como en tus proyectos).
2. **Crear el lector** dentro de `main`: `Scanner sn = new Scanner(System.in);` (`System.in` es el teclado).
3. **Leer** con el método adecuado para cada tipo de dato:

Método	Lee	Ejemplo
`sn.nextLine()`	Una línea completa de texto (con espacios)	`String nombre = sn.nextLine();`
`sn.next()`	Una sola palabra	`String palabra = sn.next();`
`sn.nextInt()`	Un número entero	`int edad = sn.nextInt();`
`sn.nextDouble()`	Un número con decimales	`double precio = sn.nextDouble();`

Saludo.java

```
import java.util.Scanner;

public class Saludo {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);     // el "lector" del teclado

        System.out.println("¿Cómo te llamas?");
        String nombre = sn.nextLine();           // lee una línea de texto

        System.out.println("¿Cuántos años tienes?");
        int edad = sn.nextInt();                 // lee un número entero

        System.out.println("Hola " + nombre + ", en 10 años tendrás " + (edad + 10));
    }
}
```

Consola (en verde, lo que escribe el usuario)

```
¿Cómo te llamas?
Ana
¿Cuántos años tienes?
19
Hola Ana, en 10 años tendrás 29
```

La trampa del Enter (aparece en todos tus menús)

Mira qué pasa si después de leer un número intentas leer un texto:

Trampa.java

```java
import java.util.Scanner;

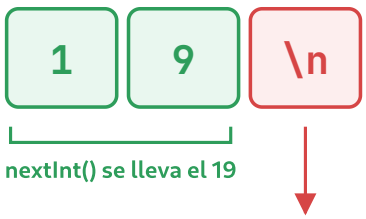
public class Trampa {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);
        System.out.println("Edad:");
        int edad = sn.nextInt();
        System.out.println("Ciudad:");
        String ciudad = sn.nextLine();     // ✘ se come el Enter que dejó nextInt
        System.out.println("Edad: " + edad + ", ciudad: [" + ciudad + "]");
    }
}
```

Consola

```
Edad:
19
Ciudad:
Edad: 19, ciudad: []
```

¡Ni siquiera te dejó escribir la ciudad! La razón:

Lo que queda guardado al escribir 19 y presionar Enter:



(\n = el Enter)

La solución

```
edad = sn.nextInt();
sn.nextLine(); // traga \n
```

Después ya puedes leer texto.

sobra el Enter: el siguiente nextLine() lo lee
y regresa "" (texto vacío). ¡Parece que se saltó la pregunta!

Hay dos formas de arreglarlo. La primera es la que usas en tus menús: un `sn.nextLine();` «de limpieza» justo después de cada `nextInt()`. La segunda, que usaste en tu proyecto de futbolistas, es leer **todo** como texto y convertirlo:

```java
int edad = Integer.parseInt(sn.nextLine());         // lee la línea y la convierte a int
double precio = Double.parseDouble(sn.nextLine());  // lo mismo, pero a double
```

1.7 Comparar textos: equals, no ==

Tu nota del 28 de agosto dice que un String no se compara con `==`, sino con `equals`. Aquí está la prueba:

Textos.java

```java
import java.util.Scanner;

public class Textos {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);
        String guardado = "hola";
        String escrito = sn.nextLine();                // el usuario escribe: hola
        System.out.println(escrito == guardado);       // ✘ compara si es el MISMO objeto
        System.out.println(escrito.equals(guardado));  // ✔ compara las letras
    }
}
```

Consola (el usuario escribe hola)

```
hola
false
true
```

Las dos dicen «hola», pero `==` da `false`. ¿Por qué? Porque `==` pregunta si son **el mismo objeto** en memoria (la misma cajita), y aquí hay dos cajitas distintas que casualmente tienen las mismas letras. `.equals()` sí compara **el contenido**.

Para comparar…	Usa	Ejemplo
`int`, `double`, `char`, `boolean`	`==`	`if (opcion == 1)`
`String`, exacto	`.equals()`	`if (id.equals("jorge@gmail.com"))`
`String`, sin importar mayúsculas	`.equalsIgnoreCase()`	`while (otro.equalsIgnoreCase("s"))`

🎯 **PARA EL EXAMEN**

¿Por qué no se usa `==` con String? Porque `String` es una clase (cada texto es un objeto) y `==` compara **referencias** (si es el mismo objeto), mientras que `equals()` compara el **contenido** (las letras).

Otros métodos útiles de String

Método	Qué hace	Con `String s = "Hola";`
`s.length()`	Cuántos caracteres tiene	`4`
`s.toUpperCase()`	Todo a mayúsculas	`"HOLA"`
`s.charAt(i)`	El carácter de la posición `i` (se cuenta desde 0)	`s.charAt(1)` es `'o'`
`s.isEmpty()`	¿Está vacío?	`false`

En resumen

- Un programa Java vive dentro de una `class` y empieza a ejecutarse en `main`.
- Toda instrucción termina en `;` y cada `{` necesita su `}`.
- Las variables tienen tipo: `int`, `double`, `boolean`, `char`, `String`.
- Entre dos `int`, `/` es división entera: `7 / 2` da `3`.
- `Scanner` lee del teclado; después de `nextInt()` va un `nextLine()` de limpieza.
- Los `String` se comparan con `.equals()`, nunca con `==`.

Prácticas de la Lección 1

Práctica 1.1 · ¿Qué imprime? SIN COMPU

Escribe lo que muestra cada `println`. Hay dos trampas.

```java
int a = 5;
int b = 2;
System.out.println(a + b);
System.out.println("Resultado: " + a + b);
System.out.println("Resultado: " + (a + b));
System.out.println(a / b);
System.out.println(5.0 / 2);
```

Práctica 1.2 · Encuentra los 6 errores SIN COMPU

El archivo se llama `Programa.java`. Encierra en un círculo cada error y escribe cómo se corrige.

```java
public class programa {
    public static void main(String[] args) {
        int edad = 19
        String nombre = 'Ana';
        system.out.println("Hola " + nombre);
        Int anio = 2026;
        System.out.println("Edad: " + edad);
    }
```

Práctica 1.3 · Preséntate A PROGRAMAR

Escribe `Presentacion.java`: pide tu nombre, tu edad y tu ciudad (¡cuidado con la trampa del Enter!) y muestra un mensaje como este:

Consola

```
Hola Jorge, tienes 19 años, vives en Guadalajara y el próximo año tendrás 20.
```

Práctica 1.4 · Ticket con IVA A PROGRAMAR

Pide el precio de un producto (`double`) y la cantidad (`int`). Muestra el subtotal, el IVA (16 %) y el total. Para comprobar: con precio 500 y cantidad 2, el total debe ser `1160.0`.

Autoevaluación: ¿ya puedes…?

- ☐ Escribir de memoria el esqueleto de un programa con `main`.
- ☐ Elegir el tipo correcto para un dato (una edad, un precio, un nombre, si algo está ocupado).
- ☐ Explicar por qué `7 / 2` da `3`.
- ☐ Leer un número y luego un texto con `Scanner` sin que se salte la pregunta.
- ☐ Explicar por qué los `String` se comparan con `equals`.

2

LECCIÓN 2 · DÍA 2

Decisiones y ciclos

Cómo elige un programa qué camino tomar y cómo repite cosas sin copiar código

Vas a aprender:

- Comparar valores y combinar condiciones
- Tomar decisiones con `if` y `switch`
- Repetir con `for`, `while` y `do-while`
- Hacer pruebas de escritorio
- Armar el menú que usas en todos tus proyectos
- Pasar de pseudocódigo a Java

2.1 Comparaciones y condiciones

Una **condición** es una pregunta que se responde con `true` o `false`. Se arma con operadores de comparación y se combina con operadores lógicos:

Operador	Significa	Ejemplo que da true
`==`	igual a	`5 == 5`
`!=`	diferente de	`5 != 3`
`>` `<`	mayor / menor que	`7 > 2`
`>=` `<=`	mayor / menor o igual	`5 >= 5`

Lógico	Se lee	Es true cuando…
`&&`	Y	las **dos** condiciones son true
`||`	O	**al menos una** es true
`!`	NO	invierte: `!true` es false

Dos ejemplos reales de tus proyectos:

```
// Futbolistas: error si el número es negativo O se pasa del tamaño O esa posición está vacía
if (jugador < 0 || jugador >= numeroJugadores || EquipoFutbol[jugador] == null)

// Clase Usuarios: para iniciar sesión deben coincidir el id Y la contraseña
if (this.id.equals(id) && this.contraseña.equals(contraseña))
```

⚠️ **CUIDADO: = NO ES ==**

Un solo `=` **asigna** (guarda un valor). Dos `==` **comparan**. Escribir `if (edad = 18)` es un error de compilación.

2.2 if, else if y else

Con `if` el programa decide si ejecuta un bloque o no. Con `else` le das un camino alternativo, y con `else if` encadenas más preguntas:

```
int calificacion = 85;

if (calificacion >= 90) {
    System.out.println("Excelente");
} else if (calificacion >= 70) {
    System.out.println("Aprobado");
} else {
    System.out.println("Reprobado");
}
```

Consola

```
Aprobado
```

Java revisa las condiciones **de arriba hacia abajo** y entra **solo al primer bloque** cuya condición sea `true`; todos los demás se los salta. Con 85, la primera condición (`85 >= 90`) es false y la segunda (`85 >= 70`) es true: imprime «Aprobado» y ya no revisa nada más. El `else` final es el «si no fue ninguna de las anteriores».

> ✅ **TIP**
> Si la variable ya es `boolean`, no necesitas compararla con `true`: `if (ocupada == true)` es lo mismo que `if (ocupada)`, y `if (ocupada == false)` es lo mismo que `if (!ocupada)`.

2.3 switch: elegir entre varias opciones

Cuando comparas **una misma variable** contra varios valores fijos (como la opción de un menú), `switch` queda más ordenado que muchos `if`:

```java
switch (menu) {
    case 1:
        System.out.println("Capturar futbolista");
        break;                      // sale del switch
    case 2:
        System.out.println("Concentrarse");
        break;
    default:                        // si no coincidió ningún case (como un else)
        System.out.println("Esa opción no existe");
}
```

- Cada `case` es un valor posible. Funciona con `int`, `char` y `String` (no con `double`).
- `break` termina el switch. **Sin él, Java sigue ejecutando los cases de abajo.**
- `default` es opcional: se ejecuta cuando ningún case coincide.
- En tus proyectos verás `case 1: { ... }` con llaves: son opcionales, solo agrupan el código del case (y permiten declarar variables adentro).

⚠️ **CUIDADO: ¿Y SI OLVIDO EL BREAK?**

```java
int opcion = 2;
switch (opcion) {
    case 1:
        System.out.println("Uno");
    case 2:
        System.out.println("Dos");      // entra aquí...
    case 3:
        System.out.println("Tres");     // ...y como no hay break, sigue de largo
    default:
        System.out.println("Otro");
}
```

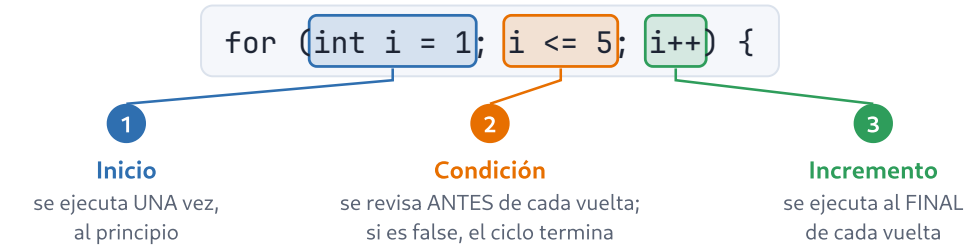
Consola

```
Dos
Tres
Otro
```

Entra en el `case 2` y, como no encuentra ningún `break`, sigue «de largo» por todos los de abajo, incluido el `default`. Es una pregunta trampa clásica de examen.

2.4 El ciclo for

Un **ciclo** repite un bloque de código. El `for` se usa cuando sabes **cuántas veces** quieres repetir. Tiene tres partes, separadas por `;`:



Orden: ① → ② → cuerpo → ③ → ② → cuerpo → ③ → … hasta que ② sea false

```java
for (int i = 1; i <= 5; i++) {
    System.out.println("Vuelta " + i);
}
```

Consola

```
Vuelta 1
Vuelta 2
Vuelta 3
Vuelta 4
Vuelta 5
```

i	¿i <= 5?	Imprime
1	sí	Vuelta 1
2	sí	Vuelta 2
3	sí	Vuelta 3
4	sí	Vuelta 4
5	sí	Vuelta 5
6	no	(termina el ciclo)

Esta tabla es una **prueba de escritorio**: seguir el código a mano, vuelta por vuelta.

> **🎯 PARA EL EXAMEN: LA PRUEBA DE ESCRITORIO**
>
> Cuando te pregunten «¿qué imprime este código?», no adivines: haz una tabla con una columna por variable y ve llenándola vuelta por vuelta, como si fueras la computadora. Así nunca te equivocas con los ciclos.

Ciclos anidados: un for dentro de otro

Así dibujaste el cuadrado y el triángulo en tu práctica de figuras. El for de adentro da **todas** sus vueltas por **cada** vuelta del de afuera:

```java
int lado = 3;
for (int j = 0; j < lado; j++) {      // filas
    for (int i = 0; i < lado; i++) {  // columnas
        System.out.print("*");
    }
    System.out.println("");           // fin de fila
}
```

Consola

```
***
***
***
```

```java
int altura = 4;
for (int i = 1; i <= altura; i++) { // fila i
    for (int j = 0; j < i; j++) {   // i veces
        System.out.print("*");
    }
    System.out.println("");
}
```

Consola

```
*
**
***
****
```

En el cuadrado, cada fila imprime `lado` asteriscos: 3 filas × 3 = 9. En el triángulo, la fila `i` imprime `i` asteriscos, porque la condición del for de adentro (`j < i`) **depende** de la fila en la que va el de afuera.

2.5 while y do-while

while: «mientras…»

Revisa la condición **antes** de cada vuelta. Si desde el inicio es false, **no entra ni una vez**.

```java
while (condicion) {
    // se repite mientras sea true
}
```

do-while: «haz… mientras…»

Primero ejecuta y **después** revisa: se ejecuta **al menos una vez**. ¡Ojo con el `;` del final!

```java
do {
    // se ejecuta al menos una vez
} while (condicion);
```

```java
int x = 10;

while (x < 5) {                  // 10 < 5 es false: ni una vuelta
    System.out.println("while: " + x);
    x++;
}

do {                             // primero hace...
    System.out.println("do-while: " + x);
    x++;
} while (x < 5);                 // ...y después pregunta
```

Consola

```
do-while: 10
```

Con `x = 10`, el `while` no entra nunca, pero el `do-while` imprime una vez antes de darse cuenta de que `10 < 5` es false.

Ciclo	Úsalo cuando…	Ejemplo
`for`	Sabes cuántas vueltas: «10 veces», «cada posición del arreglo»	Recorrer las 5 habitaciones
`while`	Repites mientras algo pase, y tal vez ni una vez	Mientras queden intentos
`do-while`	Necesitas que pase **al menos una vez**	Mostrar un menú; pedir un dato hasta que sea válido

2.6 El patrón del menú

Este esqueleto aparece en **todos** tus proyectos (futbolistas, hotel, lista de usuarios, figuras). Si lo entiendes, entiendes la mitad de cada programa:

Menu.java

```java
import java.util.Scanner;

public class Menu {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);
        int opcion;                                    // ① la variable va AFUERA del do

        do {                                           // ② se muestra al menos una vez
            System.out.println("1. Saludar");
            System.out.println("2. Despedirse");
            System.out.println("3. Salir");
            opcion = sn.nextInt();
            sn.nextLine();                             // ③ traga el Enter

            switch (opcion) {                          // ④ elige según la opción
                case 1:
                    System.out.println("¡Hola!");
                    break;
                case 2:
                    System.out.println("¡Adiós!");
                    break;
                case 3:
                    System.out.println("Saliendo...");
                    break;
                default:
                    System.out.println("Opción inválida");
            }
        } while (opcion != 3);                         // ⑤ repite hasta elegir salir
    }
}
```

1. `opcion` se declara **antes** del `do`, porque la condición del final (`while`) también la necesita.
2. Es `do-while` porque el menú debe mostrarse al menos una vez.

3. El `nextLine()` de limpieza evita la trampa del Enter si después lees texto.
4. El `switch` manda a cada opción, y `default` atrapa las opciones que no existen.
5. El ciclo se repite mientras la opción **no** sea la de salir.

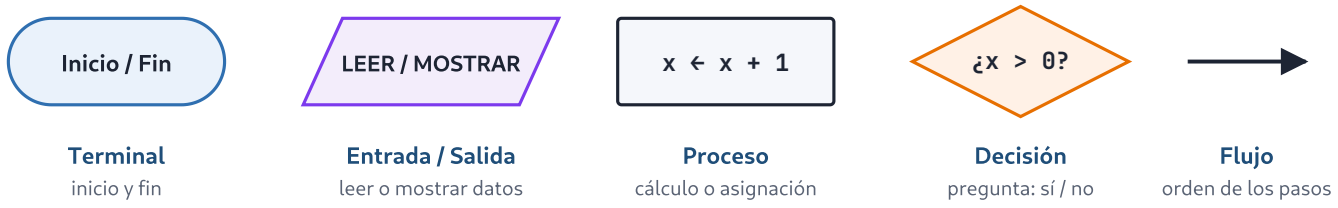
> 💡 **IDEA CLAVE: BREAK EN SWITCH VS. BREAK EN CICLO**
>
> Un `break` dentro de un `switch` sale **del switch** (el menú sigue). Un `break` dentro de un ciclo, sin switch de por medio, sale **del ciclo**. En tu proyecto de futbolistas usaste los dos: uno para cerrar el `case 1` y otro para romper el `do-while` interno cuando la plantilla ya estaba llena.

2.7 Pseudocódigo y diagramas de flujo

Antes de programar, en clase describieron sus algoritmos en **pseudocódigo** (pasos en español estructurado) y en **diagramas de flujo**. Pasar del pseudocódigo a Java es casi traducir palabra por palabra:

Pseudocódigo	Java
INICIO … FIN	`public static void main(String[] args) { … }`
MOSTRAR "Hola"	`System.out.println("Hola");`
LEER edad	`edad = sn.nextInt();`
intentos ← 0	`intentos = 0;`
SI cond ENTONCES … SI NO … FIN SI	`if (cond) { … } else { … }`
SEGÚN opcion HACER caso 1: …	`switch (opcion) { case 1: … }`
MIENTRAS cond HACER … FIN MIENTRAS	`while (cond) { … }`
PARA i ← 1 HASTA 10 HACER … FIN PARA	`for (int i = 1; i <= 10; i++) { … }`
REPETIR … HASTA QUE cond	`do { … } while (!cond);` (¡la condición se invierte!)
Y / O / NO	`&&` / `||` / `!`

Y estos son los símbolos estándar de un diagrama de flujo:



En resumen

- Las condiciones se arman con `== != > < >= <=` y se combinan con `&&`, `||` y `!`.
- `if / else if / else` entra solo al **primer** bloque verdadero.
- En un `switch`, cada `case` necesita su `break`.
- `for` cuando sabes cuántas vueltas; `while` puede no entrar nunca; `do-while` entra al menos una vez.
- Para predecir lo que imprime un ciclo, haz una prueba de escritorio.

Prácticas de la Lección 2

Práctica 2.1 · ¿Qué imprime cada uno? SIN COMPU

a)

```java
for (int i = 0; i < 3; i++) {
    System.out.print(i + " ");
}
```

b)

```java
int n = 10;
while (n > 0) {
    System.out.print(n + " ");
    n = n - 3;
}
```

c)

```java
int k = 1;
do {
    k = k * 2;
} while (k < 20);
System.out.println(k);
```

d)

```java
for (int f = 1; f <= 3; f++) {
    for (int c = 1; c <= f; c++) {
        System.out.print(c);
    }
    System.out.println();
}
```

Práctica 2.2 · El switch tramposo SIN COMPU

```java
char letra = 'b';
switch (letra) {
    case 'a':
        System.out.println("A");
    case 'b':
        System.out.println("B");
    case 'c':
        System.out.println("C");
        break;
    default:
        System.out.println("?");
}
```

¿Qué imprime? ¿Por qué?

Práctica 2.3 · Par o impar A PROGRAMAR

Pide un número entero y muestra si es par o impar, y además si es positivo, negativo o cero.

Práctica 2.4 · Tabla de multiplicar A PROGRAMAR

Pide un número y muestra su tabla del 1 al 10 usando un `for`, con el formato `7 x 1 = 7`.

Práctica 2.5 · Calculadora con menú A PROGRAMAR

Con `do-while` y `switch`, muestra el menú *1. Sumar · 2. Restar · 3. Multiplicar · 4. Dividir · 5. Salir*. Para las opciones 1 a 4 pide dos números (`double`) y muestra el resultado. Si intentan dividir entre 0, muestra un aviso en lugar de dividir. El menú se repite hasta elegir 5.

Práctica 2.6 · Del pseudocódigo a Java A PROGRAMAR

Traduce este algoritmo (basado en el inicio de sesión de su tarea de pseudocódigo):

Pseudocódigo

```
INICIO
   idCorrecto ← "admin"
   passCorrecta ← "1234"
   intentos ← 0
   acceso ← falso
   MIENTRAS intentos < 3 Y acceso = falso HACER
      MOSTRAR "Id:"
      LEER id
      MOSTRAR "Contraseña:"
      LEER pass
      SI id = idCorrecto Y pass = passCorrecta ENTONCES
         MOSTRAR "Bienvenido"
         acceso ← verdadero
      SI NO
         intentos ← intentos + 1
         MOSTRAR "Datos incorrectos"
      FIN SI
   FIN MIENTRAS
   SI acceso = falso ENTONCES
      MOSTRAR "Acceso bloqueado"
   FIN SI
FIN
```

Autoevaluación: ¿ya puedes…?

- ☐ Combinar condiciones con `&&` y `||` y explicar cuándo dan true.
- ☐ Predecir qué hace un `switch` al que le falta un `break`.
- ☐ Hacer la prueba de escritorio de un `for` (incluso uno anidado).
- ☐ Explicar la diferencia entre `while` y `do-while`.
- ☐ Escribir de memoria el esqueleto de un menú con `do-while` + `switch`.

LECCIÓN 3 · DÍA 3

3 Clases y objetos

La idea central de la POO: moldes, y las cosas que se fabrican con ellos

Vas a aprender:

- Qué es la Programación Orientada a Objetos
- La diferencia entre clase y objeto
- Atributos, métodos y constructores
- Para qué sirve `this`
- Crear objetos con `new` y usarlos
- Leer y dibujar un diagrama de clases UML

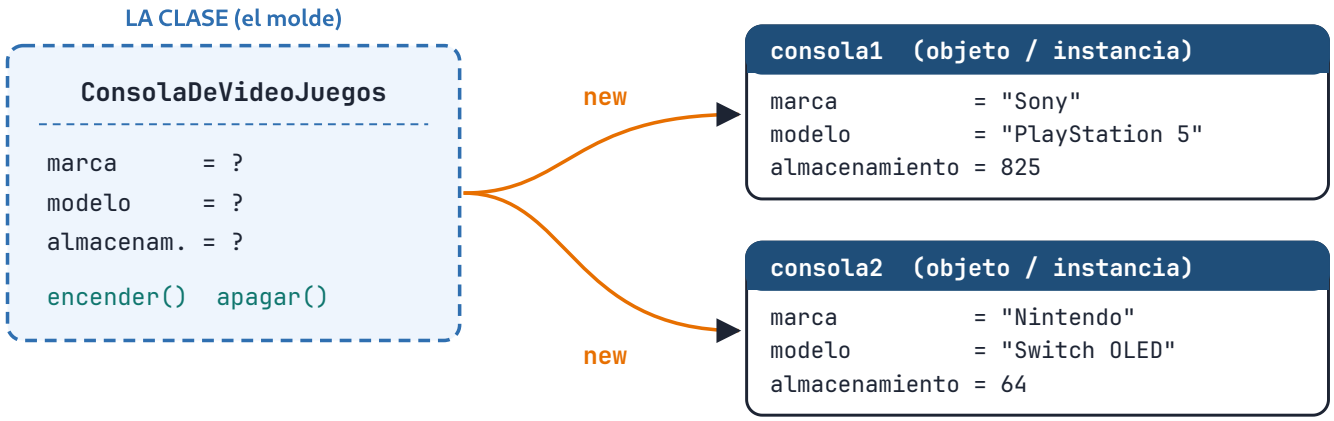
3.1 ¿Qué es la Programación Orientada a Objetos?

Según tu nota del 21 de agosto, la POO es un **paradigma** (una forma de pensar y organizar programas) en el que el programa se arma como un conjunto de **objetos** que trabajan juntos, igual que en el mundo real. Cada objeto tiene:

- **Características** (datos): un futbolista tiene nombre, edad, dorsal…
- **Comportamientos** (acciones): un futbolista puede entrenar, viajar, jugar un partido…

🧩 **ANALOGÍA: EL MOLDE DE GALLETAS**

Una **clase** es como un molde de galletas (o el plano de una casa): describe cómo será cada galleta, pero **no es** una galleta. Con un mismo molde puedes hacer muchas galletas; cada galleta es un **objeto**. Todas tienen la misma forma, pero cada una puede tener distinto relleno.



🎯 **PARA EL EXAMEN: DEFINICIONES**

- **Clase:** plantilla o molde que define los atributos y métodos que tendrán sus objetos.
- **Objeto:** elemento concreto creado a partir de una clase, con sus propios valores. También se le llama **instancia**.
- **Instanciar:** crear un objeto a partir de una clase (con `new`).
- **POO:** paradigma que organiza el programa en objetos que agrupan datos (atributos) y funciones (métodos), inspirado en el mundo real. Ayuda a crear programas más robustos, reutilizables y fáciles de mantener.

3.2 Las partes de una clase

Esta es una versión resumida de tu clase del 4 de septiembre. Fíjate en sus tres partes:

ConsolaDeVideoJuegos.java

```java
public class ConsolaDeVideoJuegos {

    // ATRIBUTOS: lo que el objeto TIENE
    private String marca;
    private String modelo;
    private int almacenamiento;

    // CONSTRUCTOR: cómo NACE el objeto
    public ConsolaDeVideoJuegos(String marca, String modelo, int almacenamiento) {
        this.marca = marca;
        this.modelo = modelo;
        this.almacenamiento = almacenamiento;
    }

    // MÉTODOS: lo que el objeto HACE
    public void encender() {
        System.out.println("Encendiendo la " + this.modelo + " de " + this.marca);
    }

    public void apagar() {
        System.out.println("Apagando la " + this.modelo + " (" + this.almacenamiento + " GB)");
    }
}
```

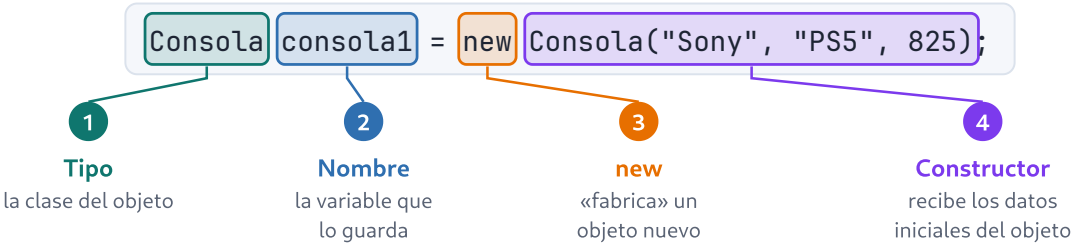
Parte	Qué es	Cómo reconocerla
Atributos	Las características: variables declaradas dentro de la clase, pero fuera de los métodos	Suelen ser **sustantivos**: `marca`, `modelo`
Constructor	Método especial que prepara al objeto cuando nace	Se llama **igual que la clase** y no tiene tipo de retorno
Métodos	Las acciones que el objeto sabe hacer	Suelen ser **verbos**: `encender()`, `apagar()`

¿Y ese `private`? Significa que solo la propia clase puede tocar ese atributo. Lo verás a fondo en la Lección 4.

> ✅ **TRUCO PARA EL EXAMEN**
>
> Si te dan un enunciado como *«Un hotel tiene habitaciones con número y precio por noche; las habitaciones se pueden reservar y liberar»*, subraya: los **sustantivos** suelen ser clases o atributos (*Habitación, número, precio*) y los **verbos** suelen ser métodos (*reservar, liberar*).

3.3 Crear y usar objetos

La clase sola no hace nada: es solo el molde. Para usarla hay que **crear objetos** con `new`. Analicemos la línea (con el nombre de la clase abreviado para que quepa):



Para pedirle algo a un objeto se usa el **punto**: `consola1.encender();` se lee «consola1, enciéndete».

Principal.java

```java
public class Principal {
    public static void main(String[] args) {
        ConsolaDeVideoJuegos consola1 = new ConsolaDeVideoJuegos("Sony", "PlayStation 5", 825);
        ConsolaDeVideoJuegos consola2 = new ConsolaDeVideoJuegos("Nintendo", "Switch OLED", 64);

        consola1.encender();      // "consola1, enciéndete"
        consola2.encender();      // "consola2, enciéndete"
        consola1.apagar();
    }
}
```

Consola

```
Encendiendo la PlayStation 5 de Sony
Encendiendo la Switch OLED de Nintendo
Apagando la PlayStation 5 (825 GB)
```

Las dos consolas salieron del mismo molde, pero **cada objeto guarda sus propios datos**: cuando le hablas a `consola2`, usa los suyos.

3.4 El constructor

Como dice tu nota del 1 de septiembre, el constructor es un método especial que sirve para **inicializar** el objeto (darle sus valores iniciales). Sus reglas:

1. Se llama **exactamente igual** que la clase.
2. **No tiene tipo de retorno**, ni siquiera `void`. Así lo distingues de un método normal.
3. Se ejecuta **automáticamente** cada vez que haces `new`.

Si no escribes ningún constructor, Java pone uno invisible **por defecto**, sin parámetros y vacío: `public Clase() { }`. Por eso a veces puedes hacer `new Clase()` sin haber escrito nada.

⚠️ **TRAMPA DE EXAMEN**

Si escribes un constructor **con parámetros**, Java **ya no** crea el vacío. Con la clase de arriba, `new ConsolaDeVideoJuegos()` da error de compilación. Si quieres las dos formas, escribe los dos constructores, como hiciste en tu clase `Usuario`:

Usuario.java

```java
public class Usuario {
    private String correo;
    private String contraseña;

    public Usuario() {                                      // constructor vacío
    }

    public Usuario(String correo, String contraseña) {      // constructor con parámetros
        this.correo = correo;
        this.contraseña = contraseña;
    }
}
```

```java
Usuario u1 = new Usuario();                                  // usa el vacío
Usuario u2 = new Usuario("jorge@gmail.com", "Clave#2026");   // usa el de 2 parámetros
```

Tener varios constructores con distintos parámetros es un caso de **sobrecarga** (lo verás en la Lección 6).

3.5 La palabra this

En el constructor, el parámetro y el atributo se llaman igual (`marca`). ¿Cómo sabe Java cuál es cuál? Con `this`: `this.marca` es «el atributo marca de **este** objeto», y `marca` sola es el parámetro (el dato que llegó).

🧩 **ANALOGÍA**

`this` es como decir «mi»: `this.marca = marca;` se lee «**mi** marca es la marca que me pasaron».

¿Qué pasa si se te olvida?

Consola.java

```java
public class Consola {
    private String marca;
    private int almacenamiento;

    public Consola(String marca, int almacenamiento) {
        marca = marca;                      // ✘ el parámetro se asigna a sí mismo
        almacenamiento = almacenamiento;    // ✘ el atributo nunca se toca
    }

    public void mostrar() {
        System.out.println("Marca: " + this.marca);
        System.out.println("Almacenamiento: " + this.almacenamiento + " GB");
    }
}
```

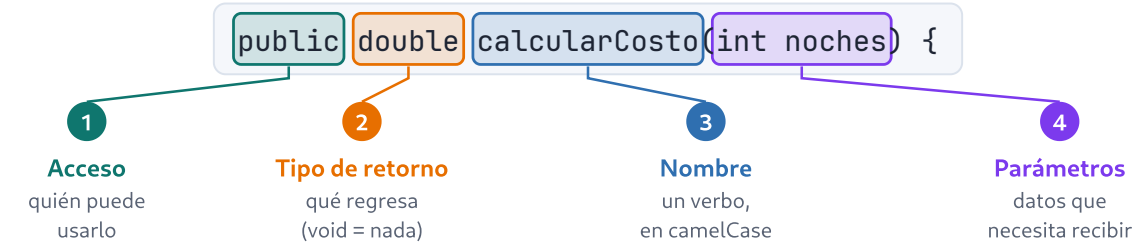
Consola

```
Marca: null
Almacenamiento: 0 GB
```

Sin `this`, Java entiende «guarda el parámetro en el parámetro» y el atributo se queda con su **valor por defecto**:

Tipo del atributo	Valor por defecto
`int`	`0`
`double`	`0.0`
`boolean`	`false`
`String` (y cualquier objeto)	`null` («aquí no hay nada»)

3.6 Métodos: acciones que reciben y regresan datos




```java
public double calcularCosto(int noches) {
    return this.precioPorNoche * noches;     // regresa un double
}
```

- Si el método **regresa** algo, debe usar `return` con un valor del tipo indicado (aquí, `double`).
- Si no regresa nada, su tipo es `void`: solo **hace** algo, como imprimir.
- Lo que regresa un método se puede guardar en una variable o usar directamente:

```java
Habitacion hab = new Habitacion(101, 800.0);
double total = hab.calcularCosto(3);      // el 3 viaja al parámetro "noches"
System.out.println("Total: $" + total);
```

Consola

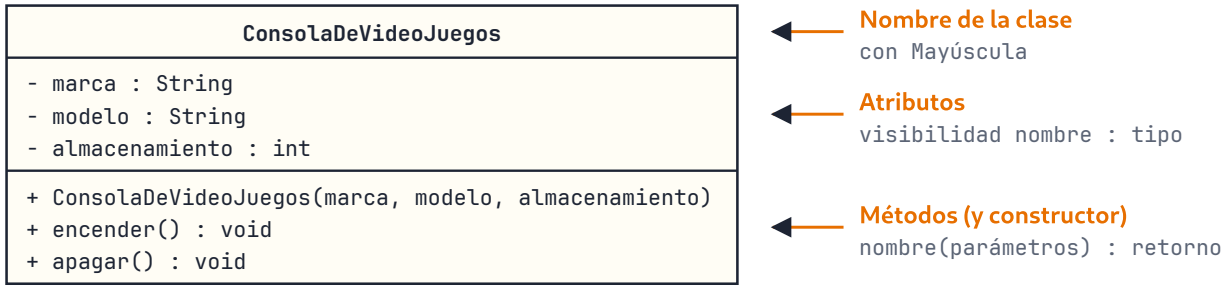
```
Total: $2400.0
```

Parámetro es la variable que aparece en la definición (`noches`); **argumento** es el valor real que mandas al llamarlo (`3`).

> 💡 **IDEA CLAVE: DEFINIR NO ES LO MISMO QUE LLAMAR**
>
> Un método se **define** una vez dentro de la clase (con su tipo, nombre y llaves) y se **llama** las veces que quieras desde un objeto: `hab.calcularCosto(3)`. Si nunca lo llamas, nunca se ejecuta.

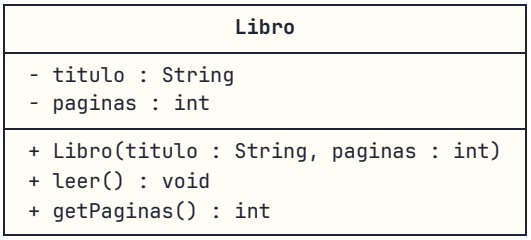
3.7 El diagrama de clases (UML)

UML (Lenguaje Unificado de Modelado) es un conjunto de dibujos estándar para diseñar programas antes de escribirlos. En un **diagrama de clases**, cada clase es un rectángulo con tres secciones, como el que hicieron en equipo para la clase `Usuario`:



Visibilidad: `+` public `-` private `#` protected `~` sin modificador (paquete)

Del diagrama al código



Libro.java

```java
public class Libro {
    private String titulo;
    private int paginas;

    public Libro(String titulo, int paginas) {
        this.titulo = titulo;
        this.paginas = paginas;
    }

    public void leer() {
        System.out.println("Leyendo " + titulo);
    }

    public int getPaginas() {
        return paginas;
    }
}
```

Fíjate en la traducción: `- titulo : String` se vuelve `private String titulo;` (en UML el tipo va **después**, tras los dos puntos; en Java va **antes**), y `+ getPaginas() : int` se vuelve `public int getPaginas()`.

En resumen

- Una **clase** es el molde; un **objeto** (instancia) es lo que se fabrica con `new`.
- Una clase tiene **atributos** (lo que tiene), **constructor** (cómo nace) y **métodos** (lo que hace).
- El constructor se llama igual que la clase y no tiene tipo de retorno.
- Si escribes un constructor con parámetros, el constructor vacío por defecto desaparece.
- `this.atributo` distingue el atributo del parámetro.
- Un método no `void` debe regresar un valor con `return`.

Prácticas de la Lección 3

Práctica 3.1 · Del enunciado al diagrama SIN COMPU

«Una mascota tiene nombre, especie y edad. Puede comer, dormir y cumplir años (cumplir años aumenta su edad en 1).» Identifica los atributos y los métodos, y dibuja su diagrama de clases (incluye el constructor).

Práctica 3.2 · Programa tu Mascota A PROGRAMAR

Escribe la clase `Mascota` de la práctica anterior. `comer()` imprime «Firulais está comiendo», `dormir()` imprime algo parecido, `cumplirAnios()` aumenta la edad y lo anuncia, y `mostrarDatos()` imprime todo. En `main`, crea dos mascotas y llama a sus métodos.

Práctica 3.3 · ¿Qué imprime? SIN COMPU

Foco.java

```java
public class Foco {
    private boolean encendido;
    private String color;

    public Foco(String color) {
        this.color = color;
    }

    public void presionar() {
        encendido = !encendido;     // ! invierte: true ↔ false
    }

    public void estado() {
        System.out.println(color + ": " + encendido);
    }
}
```

```java
Foco f1 = new Foco("Rojo");
Foco f2 = new Foco("Azul");
f1.presionar();
f2.presionar();
f2.presionar();
f1.estado();
f2.estado();
```

Pista: ¿con qué valor empieza un `boolean` que nadie inicializó?

Práctica 3.4 · Encuentra los 4 errores SIN COMPU

Celular.java

```java
public class Celular {
    private String marca;
    private int bateria;

    public void Celular(String marca, int bateria) {
        this.marca = marca;
        this.bateria = bateria;
    }

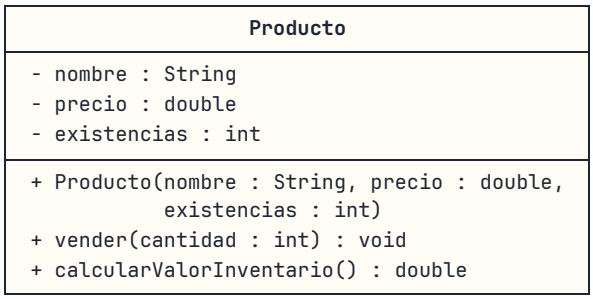
    public int getBateria() {
        System.out.println(bateria);
    }

    public void cargar() {
        bateria = 100
    }
}
```

Prueba.java

```java
public class Prueba {
    public static void main(String[] args) {
        Celular c = new Celular("Samsung", 80);
        c.apagar();
    }
}
```

Práctica 3.5 · Del diagrama al código A PROGRAMAR



Escribe la clase. `vender()` resta la cantidad a las existencias solo si alcanzan (si no, avisa). `calcularValorInventario()` regresa precio × existencias. Pruébala en un `main`.

Autoevaluación: ¿ya puedes…?

- ☐ Explicar con tus palabras la diferencia entre clase y objeto.
- ☐ Señalar en una clase sus atributos, su constructor y sus métodos.
- ☐ Escribir un constructor con `this` y crear objetos con `new`.
- ☐ Escribir un método que regrese un valor y usar ese valor.
- ☐ Convertir un diagrama de clases en código (y al revés).

LECCIÓN 4 · DÍA 4

4 Encapsulamiento y static

Proteger los datos de un objeto y entender qué le pertenece a la clase

Vas a aprender:

- Qué es el encapsulamiento y para qué sirve
- Los cuatro modificadores de acceso
- Escribir getters y setters con validación
- Qué significa `static`
- Por qué `main` es `static`
- Crear constantes con `final`

4.1 El problema: datos sin protección

Main.java

```
class CuentaBancaria {
    public double saldo;          // ✘ cualquiera lo puede cambiar
}

public class Main {
    public static void main(String[] args) {
        CuentaBancaria cuenta = new CuentaBancaria();
        cuenta.saldo = -5000;     // nadie lo impide...
        System.out.println("Saldo: " + cuenta.saldo);
    }
}
```

Consola

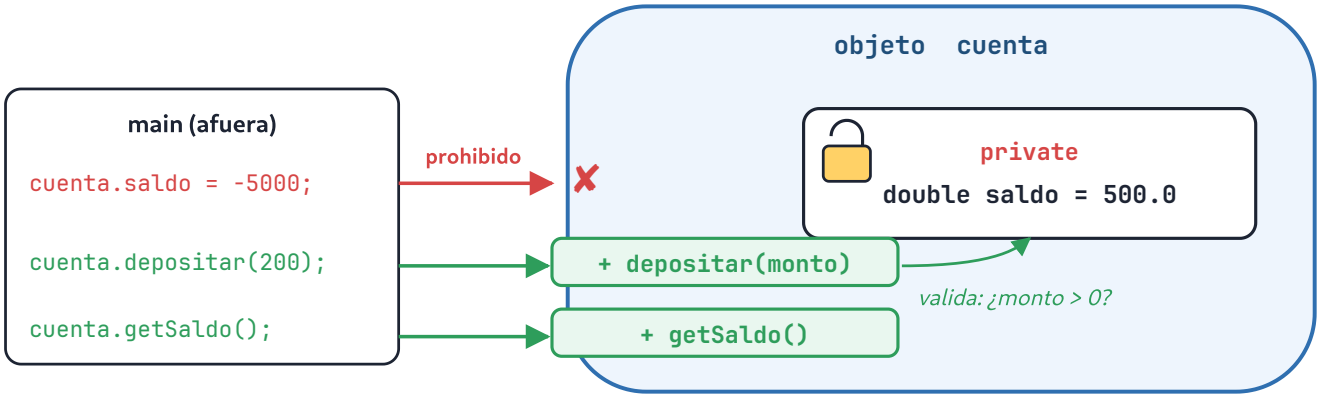
```
Saldo: -5000.0
```

Como `saldo` es `public`, cualquier parte del programa puede ponerle cualquier valor, aunque no tenga sentido. El objeto no se puede defender.

El **encapsulamiento** resuelve esto: consiste en **ocultar los datos internos de un objeto** (atributos `private`) y permitir usarlos **solo a través de sus métodos**, que pueden revisar que los valores sean válidos. Como dice tu nota del 25 de agosto, garantiza la **integridad** de los datos y esconde los detalles internos, así puedes mejorar el código por dentro sin afectar a quien lo usa.

🧩 **ANALOGÍA: EL CAJERO AUTOMÁTICO**

No puedes meter la mano a la caja fuerte del cajero (`private`). Solo puedes usar sus botones (métodos `public`), y el cajero revisa que tengas saldo antes de darte dinero.



4.2 Modificadores de acceso

Deciden **quién** puede usar un atributo, un método o una clase. Esta es la tabla que vieron en clase, con su símbolo en UML:

Modificador	Misma clase	Mismo paquete	Subclase*	Todos	UML
`public`	Sí	Sí	Sí	Sí	+
`protected`	Sí	Sí	Sí	No	#
sin modificador (*default*)	Sí	Sí	No	No	~
`private`	Sí	No	No	No	-

* Clase hija que está en **otro** paquete. Un **paquete** es como una carpeta de clases: la línea `package com.mycompany.hotelapp;` de tus proyectos dice en qué paquete está cada clase.

> 💡 **REGLA PRÁCTICA**
>
> - **Atributos:** `private`. Si vas a tener clases hijas que los necesiten, `protected` (como `Persona` y `Habitacion` en tus proyectos).
> - **Métodos y constructores:** `public`, para que se puedan usar desde afuera.

4.3 Getters y setters

Si los atributos son privados, ¿cómo los lees o los cambias desde `main`? Con métodos públicos especiales. Tus propios comentarios en `Persona.java` lo explican muy bien:

- **Getter: devuelve** el valor. Se llama `get` + Atributo, no recibe parámetros y regresa el tipo del atributo.
- **Setter: cambia** el valor. Se llama `set` + Atributo, recibe el nuevo valor y es `void`. Puede **validar** el dato antes de guardarlo.

Persona.java

```java
public class Persona {
    private String nombre;                // privados: nadie los toca directo
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
```

```
        this.edad = edad;
    }

    // GETTER: regresa el valor (no recibe nada)
    public String getNombre() {
        return this.nombre;
    }

    // SETTER: cambia el valor (recibe el nuevo, no regresa nada)
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getEdad() {
        return this.edad;
    }

    public void setEdad(int edad) {
        if (edad >= 0) {                    // ✔ valida antes de guardar
            this.edad = edad;
        } else {
            System.out.println("Edad inválida: no se puede guardar un valor negativo.");
        }
    }
}
```

```
Persona p = new Persona("Ana", 20);
p.setEdad(-5);                                  // intento con dato malo
System.out.println(p.getNombre() + " tiene " + p.getEdad() + " años");
p.setEdad(21);                                  // dato correcto
System.out.println(p.getNombre() + " tiene " + p.getEdad() + " años");
```

Consola

```
Edad inválida: no se puede guardar un valor negativo.
Ana tiene 20 años
Ana tiene 21 años
```

El setter rechazó el −5 y la edad se quedó en 20. Eso sería imposible de controlar si `edad` fuera `public`.

✅ TIPS

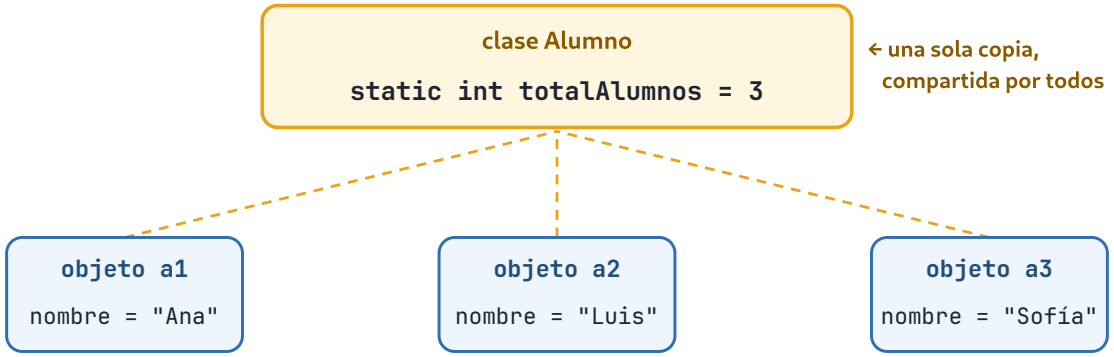
- Si un atributo tiene getter pero **no** setter, queda de **solo lectura**: en tu hotel existe `getNumero()` pero no `setNumero()`, porque el número de una habitación no debería cambiar.
- Para atributos `boolean`, el getter suele llamarse `is`…: `isOcupada()`.
- NetBeans los escribe por ti: clic derecho → **Insert Code…** (Alt+Insert) → **Getter and Setter**.

🎯 PARA EL EXAMEN

¿Cómo se logra el encapsulamiento en Java? Declarando los atributos como `private` y dando acceso a ellos mediante métodos `public` (getters y setters), que pueden validar los datos.

4.4 static: lo que pertenece a la clase

Normalmente, cada objeto tiene **su propia copia** de cada atributo (cada consola tiene su marca). Un atributo `static` es diferente: existe **una sola copia**, que pertenece a la **clase** y que comparten todos los objetos.



cada objeto tiene SU propio nombre, pero todos ven el MISMO totalAlumnos

Alumno.java

```java
public class Alumno {
    private String nombre;                    // cada objeto tiene el suyo
    private static int totalAlumnos = 0;      // UNO solo, de la clase

    public Alumno(String nombre) {
        this.nombre = nombre;
        totalAlumnos++;                       // cada new suma 1 al contador compartido
    }

    public static int getTotalAlumnos() {     // método static
        return totalAlumnos;
    }
}
```

```java
System.out.println("Alumnos: " + Alumno.getTotalAlumnos());   // se usa con el nombre de la CLASE
Alumno a1 = new Alumno("Ana");
Alumno a2 = new Alumno("Luis");
Alumno a3 = new Alumno("Sofía");
System.out.println("Alumnos: " + Alumno.getTotalAlumnos());
```

Consola

```
Alumnos: 0
Alumnos: 3
```

Un **método** `static` también es de la clase: se llama con el nombre de la clase, **sin crear ningún objeto**. Ya has usado varios:

Llamada	Qué hace
`Integer.parseInt("10")`	Convierte el texto `"10"` al `int` `10`
`Math.sqrt(25)`	Raíz cuadrada: `5.0`
`Math.pow(2, 3)`	Potencia: `8.0`
`Math.PI`	La constante π = 3.14159…

Y ahora sí: `main` es `static` porque Java necesita ejecutarlo **antes de que exista cualquier objeto**.

⚠️ ERROR CLÁSICO: «NON-STATIC METHOD … CANNOT BE REFERENCED FROM A STATIC CONTEXT»

Desde un método `static` (como `main`) no puedes usar directamente atributos o métodos que **no** son static, porque no hay un objeto: ¿de cuál consola sería la `marca`? Soluciones: crear un objeto y usarlo, o, si de verdad es algo general, hacerlo `static`.

App.java

```java
public class App {
    public void saludar() {
        System.out.println("Hola");
    }

    public static void main(String[] args) {
        saludar();              // ✘ error: saludar() no es static
        App app = new App();
        app.saludar();          // ✔ con un objeto sí
    }
}
```

Constantes con final

`final` significa «no se puede cambiar». Juntando `static final` creas **constantes**, que por convención se escriben en MAYÚSCULAS: `public static final double IVA = 0.16;`. Si después intentas `IVA = 0.20;`, el compilador no te deja.

En resumen

- **Encapsulamiento** = atributos `private` + métodos `public` que controlan el acceso.
- `public` (+) todos · `protected` (#) paquete e hijas · *default* (~) paquete · `private` (-) solo la clase.
- El getter regresa el valor; el setter lo cambia y puede validarlo.
- Lo `static` pertenece a la clase: una sola copia compartida, se usa con `Clase.miembro`.
- Desde `main` (static) solo usas lo no-static a través de un objeto.

Prácticas de la Lección 4

Práctica 4.1 · Cuenta bancaria A PROGRAMAR

Escribe la clase `CuentaBancaria` con los atributos privados `titular` (`String`) y `saldo` (`double`):

- Constructor con titular y saldo inicial (si el saldo inicial es negativo, que empiece en 0).
- `depositar(monto)`: solo suma si el monto es mayor que 0.
- `retirar(monto)`: solo resta si el monto es mayor que 0 y hay saldo suficiente; si no alcanza, imprime «Fondos insuficientes».
- `getSaldo()` y `getTitular()`, pero **sin** `setSaldo()`. ¿Por qué crees que no conviene tenerlo?

Pruébala en `main` con depósitos y retiros (incluyendo uno que no alcance).

Práctica 4.2 · ¿Compila o no? SIN COMPU

Las dos clases están en el mismo paquete. Marca cada línea de `a)` a `f)` con ✔ o ✘ y explica por qué.

Caja.java

```java
public class Caja {
    private int codigo = 1234;
    public String color = "rojo";
    protected int ancho = 10;
    int peso = 5;          // sin modificador

    private void abrir() { }
    public void cerrar() { }
}
```

Prueba.java

```java
public class Prueba {     // mismo paquete
    public static void main(String[] args) {
        Caja c = new Caja();
        c.codigo = 0;          // a)
        c.color = "azul";      // b)
        c.ancho = 20;          // c)
        c.peso = 7;            // d)
        c.abrir();             // e)
        c.cerrar();            // f)
    }
}
```

Práctica 4.3 · ¿Qué imprime? SIN COMPU

```java
class Contador {
    static int compartido = 0;
    int propio = 0;

    Contador() {
        compartido++;
        propio++;
    }
}

public class Main {
    public static void main(String[] args) {
        Contador c1 = new Contador();
        Contador c2 = new Contador();
        Contador c3 = new Contador();
        System.out.println(c3.propio + " " + Contador.compartido);
    }
}
```

Práctica 4.4 · Mejora tu Mascota A PROGRAMAR

Toma tu clase `Mascota` de la Lección 3 y agrégale: getters y setters para sus atributos privados; un `setEdad()` que no acepte edades negativas; y un contador `static` llamado `totalMascotas`, con un método `static getTotalMascotas()`, que cuente cuántas mascotas se han creado.

Autoevaluación: ¿ya puedes…?

- ☐ Definir encapsulamiento y decir cómo se logra en Java.
- ☐ Llenar de memoria la tabla de modificadores de acceso con sus símbolos UML.
- ☐ Escribir getters y setters, con un setter que valide.
- ☐ Explicar qué cambia cuando un atributo es `static`.
- ☐ Arreglar el error «non-static … cannot be referenced from a static context».

LECCIÓN 5 · DÍA 5

5 Herencia

Reutilizar código: clases hijas que reciben todo lo de su clase padre

Vas a aprender:

- La relación «es un»
- `extends` y qué se hereda
- `super(...)` para llamar al constructor del padre
- En qué orden se ejecutan los constructores
- Sobrescribir métodos y reusar los del padre
- Dibujar la herencia en UML

5.1 La idea: «es un»

Imagina que tu sistema de fútbol necesita futbolistas, entrenadores y árbitros. Los tres tienen nombre, apellido, edad e id, y los tres pueden viajar. ¿Escribirías esos atributos y métodos tres veces? Con herencia los escribes **una sola vez** en una clase **padre** (`Persona`) y las demás los **heredan**.

Clase padre	Clase hija
También se le dice **superclase** o clase base	También se le dice **subclase** o clase derivada
Lo general: `Persona` , `Habitacion` , `FigurasGeometricas`	Lo específico: `Futbolista` , `Suite` , `Circulo`

💡 **LA PRUEBA DEL «ES UN»**

Usa herencia solo si la frase «**[hija] es un(a) [padre]**» tiene sentido: un Futbolista **es una** Persona ✔; una Suite **es una** Habitación ✔; un Círculo **es una** Figura Geométrica ✔. Pero un Auto **no es** un Motor: un auto **tiene un** motor ✘. Cuando algo «tiene un», no es herencia: es un atributo.

5.2 extends: declarar la herencia

Como en la diapositiva de clase: `class Hija extends Padre { ... }`. Así se ve en tu proyecto de futbolistas (resumido):

Persona.java

```java
public class Persona {
    protected String nombre;
    protected String apellido;
    protected int edad;
    protected int id;

    Persona(String nombre, String apellido,
            int edad, int id) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.edad = edad;
        this.id = id;
    }

    public void viajar() { ... }
    public String getNombre() { ... }
    // ...más getters y setters
}
```

Futbolista.java

```java
public class Futbolista extends Persona {
    // solo lo PROPIO del futbolista:
    private int dorsal;
    private String demarcación;

    public Futbolista(
        String nombre, String apellido,
        int id, int edad,
        int dorsal, String demarcación) {
        super(nombre, apellido, id, edad);
        this.dorsal = dorsal;
        this.demarcación = demarcación;
    }

    public void jugarPartido() { ... }
    public void entrenar() { ... }
}
```

Un objeto `Futbolista` lo tiene **todo**: sus 2 atributos propios más los 4 heredados. Y puede usar los métodos de las dos clases: `f.entrenar()` (suyo), pero también `f.viajar()` y `f.getNombre()` (heredados), aunque no aparezcan escritos en `Futbolista.java`.

¿Qué se hereda?	
Atributos y métodos `public` y `protected`	**Sí**
Lo `private` del padre	**No se puede usar directamente**
Los constructores	**No: cada clase tiene los suyos**

Lo `private` sí existe dentro del objeto hijo, pero el hijo no puede tocarlo por su nombre: tiene que usar los getters y setters públicos del padre. Por eso en tus proyectos el padre usa `protected`.

> 💡 **HERENCIA SIMPLE**
>
> En Java una clase solo puede tener **un** padre: `class A extends B, C` ✘. Pero sí se vale en cadena: `Animal → Mamifero → Perro`. Además, toda clase hereda (aunque no lo escribas) de la clase `Object`; de ahí viene `toString()`, que verás en la Lección 7.

5.3 super: llamar al constructor del padre

Los atributos heredados los inicializa **el constructor del padre**. Por eso, lo primero que hace el constructor del hijo es mandarle esos datos con `super(...)`:

- `super(...)` se escribe en la **primera línea** del constructor del hijo.[1]
- Los argumentos van en el **mismo orden y tipo** que los parámetros del constructor del padre.
- Si el padre **no** tiene constructor vacío, el hijo está **obligado** a llamar a `super(...)` con datos; si no, hay error de compilación.

> 🔎 **DETECTIVE: HAY UN ERROR ESCONDIDO EN TU PROYECTO DE FUTBOLISTAS**
>
> Compara el orden de los parámetros:

```
Persona(String nombre, String apellido, int edad, int id)   // el padre espera: edad, id
super(nombre, apellido, id, edad);                          // el hijo manda:   id, edad
```

> ¡Están al revés! Como los dos son `int`, el compilador no puede darse cuenta. Mira lo que pasa con tus clases originales, sin cambiarles nada:

```
Futbolista f = new Futbolista("Leo", "Messi", 7001, 37, 10, "Delantero");
// o sea: nombre "Leo", apellido "Messi", id 7001, edad 37, dorsal 10, "Delantero"
System.out.println("ID: " + f.getId());
System.out.println("Edad: " + f.getEdad());
```

Consola

```
ID: 37
Edad: 7001
```

> Messi tendría **7001 años**. Tu programa no lo notaba porque creabas los futbolistas con ceros (`new Futbolista("","",0,0,0,"")`) y luego llenabas los datos con los setters. La corrección es `super(nombre, apellido, edad, id);`

5.4 El orden de los constructores

```java
class A {
    A() { System.out.println("Constructor de A"); }
}
class B extends A {
    B() { System.out.println("Constructor de B"); }
}
class C extends B {
    C() { System.out.println("Constructor de C"); }
}

public class Main {
    public static void main(String[] args) {
        C objeto = new C();
    }
}
```

Consola

```
Constructor de A
Constructor de B
Constructor de C
```

Al crear un `C`, primero se construye la parte de `A`, luego la de `B` y al final la de `C`: como una casa, primero los cimientos, luego las paredes y al final el techo.

Aquí nadie escribió `super()`: cuando no lo escribes, Java pone uno invisible **sin argumentos** en la primera línea del constructor.

5.5 Sobrescribir métodos y reusar los del padre

Una hija puede **redefinir** un método que heredó, escribiéndolo otra vez con el mismo nombre y los mismos parámetros. Eso se llama **sobrescribir** y se marca con `@Override`. En tu hotel, cada tipo de habitación sobrescribe `mostrarInfo()`… pero repitiendo en cada hija las mismas líneas del padre. Con `super.mostrarInfo()` puedes reusar lo que el padre ya hace y solo agregar lo nuevo:

HabitacionDoble.java (mejorada)

```java
@Override
public void mostrarInfo() {
    System.out.println("[Habitacion Doble]");
    super.mostrarInfo();                    // reusa lo que ya hace el padre
    System.out.println("  Camas dobles: " + this.camasDobles);
}
```

Consola

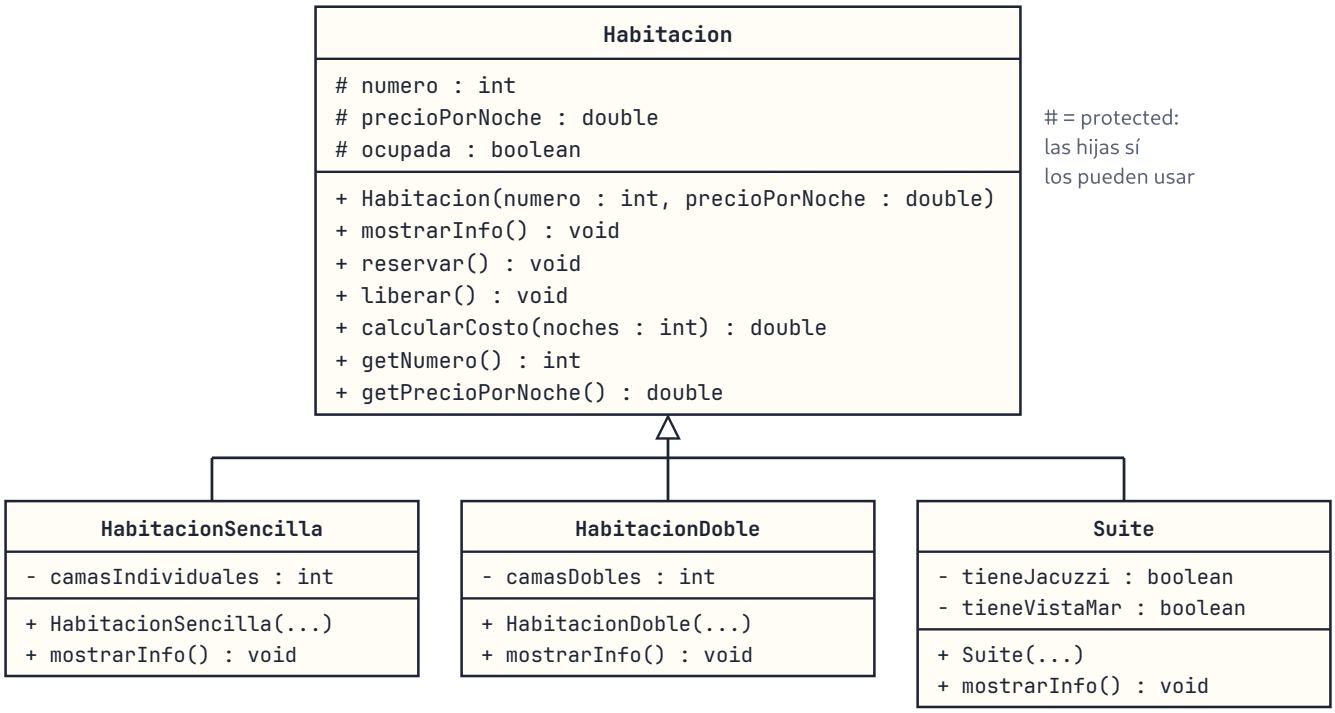
```
[Habitacion Doble]
Habitacion #201
  Precio por noche: $1500.0
  Estado: Libre
  Camas dobles: 2
```

La salida es idéntica a la de tu versión original, pero con menos código. Y si mañana cambias cómo se muestra el precio, lo cambias una sola vez, en `Habitacion`.

Palabra	Significa	Ejemplo
`this`	este objeto	`this.dorsal = dorsal;`
`super(...)`	el **constructor** del padre	`super(numero, precioPorNoche);`
`super.metodo()`	la versión del **padre** de un método	`super.mostrarInfo();`

5.6 La herencia en el diagrama de clases

En UML, la herencia se dibuja con una línea que termina en un **triángulo hueco** que apunta al **padre**. Este es el diagrama de tu práctica del hotel, redibujado para que se lea mejor:



Fíjate en los detalles: los atributos del padre llevan `#` (`protected`) para que las hijas los puedan usar, y las hijas solo muestran lo **nuevo** (sus atributos propios y lo que sobrescriben), porque lo demás se entiende heredado.

En resumen

- Herencia = una clase hija reutiliza lo de su padre: `class Hija extends Padre`.
- Úsala solo si se cumple «la hija **es un** padre».
- Se heredan atributos y métodos `public` / `protected`; los constructores no.
- `super(...)` va primero en el constructor del hijo, con los datos en el orden del padre.
- Los constructores se ejecutan del padre hacia el hijo.
- Sobrescribir = redefinir un método heredado (`@Override`); `super.metodo()` reusa la versión del padre.

[1] Es la regla clásica y la que esperan en un examen. Desde Java 25 se permiten algunas instrucciones sencillas antes de `super(...)`, pero lo normal sigue siendo escribirlo primero.

Prácticas de la Lección 5

Práctica 5.1 · ¿Herencia o no? SIN COMPU

Para cada pareja, decide si la relación es «**es un**» (herencia: di quién es el padre) o «**tiene un**» (atributo).

Pareja	¿Es un / tiene un?	Pareja	¿Es un / tiene un?
Perro – Animal		Celular – Batería	
Auto – Motor		Moto – Vehículo	
Gerente – Empleado		Escuela – Salón	

Práctica 5.2 · ¿Qué imprime? SIN COMPU

```java
class Vehiculo {
    protected String marca;

    public Vehiculo(String marca) {
        this.marca = marca;
        System.out.println("Vehículo creado: " + marca);
    }

    public void avanzar() {
        System.out.println("El vehículo avanza");
    }
}

class Moto extends Vehiculo {
    public Moto(String marca) {
        super(marca);
        System.out.println("Moto lista");
    }

    @Override
    public void avanzar() {
        super.avanzar();
        System.out.println("La moto " + marca + " acelera");
    }
}

public class Main {
    public static void main(String[] args) {
        Moto m = new Moto("Italika");
        m.avanzar();
    }
}
```

Práctica 5.3 · Encuentra los 3 errores SIN COMPU

```java
class Animal {
    private String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public void hacerSonido() {
        System.out.println("...");
    }
}

class Perro extends Animal {
    private String raza;

    public Perro(String nombre, String raza) {
        this.raza = raza;
    }

    public void mostrar() {
        System.out.println(nombre + " es de raza " + raza);
    }

    @Override
    public void ladrar() {
        System.out.println("Guau");
    }
}
```

Práctica 5.4 · Vehículos A PROGRAMAR

Crea la clase padre `Vehiculo` (atributos `marca`, `modelo`, `anio` y el método `mostrarInfo()`) y dos hijas: `Auto`, que agrega `numeroPuertas`, y `Camion`, que agrega `capacidadToneladas`. Cada hija usa `super(...)` en su constructor y sobrescribe `mostrarInfo()` reutilizando `super.mostrarInfo()`. En `main`, crea un auto y un camión y muestra su información. Dibuja también el diagrama de clases.

Autoevaluación: ¿ya puedes...?

- ☐ Decidir si dos clases deben relacionarse con herencia.
- ☐ Escribir una clase hija con `extends` y un constructor que use `super(...)`.
- ☐ Predecir el orden en que se ejecutan los constructores.
- ☐ Explicar la diferencia entre `this`, `super(...)` y `super.metodo()`.
- ☐ Dibujar la herencia en un diagrama de clases.

LECCIÓN 6 · DÍA 6

6 Abstracción y polimorfismo

Clases abstractas, sobrecarga contra sobrescritura, y un mismo mensaje con muchas respuestas

Vas a aprender:

- Sobrescritura y para qué sirve `@Override`
- Sobrecarga de métodos
- La diferencia entre las dos (pregunta clásica)
- Clases y métodos abstractos
- Qué es el polimorfismo y por qué es tan útil
- Los 4 pilares de la POO

6.1 Sobrescritura (@Override)

Ya la usaste en la lección anterior: una clase hija **redefine** un método heredado, con el **mismo nombre, los mismos parámetros y el mismo tipo de retorno**. Cada figura de tu práctica sobrescribe `area()` a su manera.

La anotación `@Override` es opcional, pero muy recomendable: le pide al compilador que **verifique** que de verdad estás sobrescribiendo algo.

> ⚠️ **¿POR QUÉ USAR @OVERRIDE?**
>
> Si te equivocas y escribes `public void Area()` (con mayúscula), **sin** `@Override` Java crea en silencio un método nuevo y tu programa sigue usando el del padre, sin avisarte. **Con** `@Override`, el compilador marca el error de inmediato: *method does not override or implement a method from a supertype*.

6.2 Sobrecarga

Sobrecargar es tener **varios métodos con el mismo nombre** en la misma clase, pero con **parámetros diferentes** (en cantidad, tipo u orden). Java elige cuál usar según los argumentos que le mandes:

```java
class Calculadora {
    public int sumar(int a, int b) {
        return a + b;
    }
    public int sumar(int a, int b, int c) {          // otro número de parámetros
        return a + b + c;
    }
    public double sumar(double a, double b) {        // otro tipo de parámetros
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculadora calc = new Calculadora();
        System.out.println(calc.sumar(2, 3));        // usa el de (int, int)
        System.out.println(calc.sumar(2, 3, 4));     // usa el de (int, int, int)
        System.out.println(calc.sumar(2.5, 1.5));    // usa el de (double, double)
    }
}
```

Consola

```
5
9
4.0
```

Ya usaste sobrecarga sin saberlo: los dos constructores de `Usuario`, y el mismo `System.out.println()`, que acepta `int`, `double`, `String`…

> ⚠️ **CUIDADO**
> Cambiar **solo** el tipo de retorno no es sobrecarga: `int calcular()` y `double calcular()` en la misma clase dan error. Lo que debe cambiar son los **parámetros**.

Sobrecarga contra sobrescritura (pregunta casi segura)

	Sobrecarga (*overloading*)	Sobrescritura (*overriding*)
¿Dónde?	En la misma clase	En una clase hija
Nombre	El mismo	El mismo
Parámetros	**Diferentes**	**Iguales**
¿Necesita herencia?	No	Sí
Anotación	—	`@Override`
¿Cuándo elige Java?	Al compilar, por los argumentos	Al ejecutar, por el objeto real
Ejemplo tuyo	`Usuario()` y `Usuario(correo, contraseña)`	`area()` en Cuadrado, Triangulo y Circulo

6.3 Abstracción y clases abstractas

La **abstracción** consiste en quedarte con lo **esencial** de algo e ignorar los detalles: definir **qué** debe hacer sin decir todavía **cómo**. Piensa en tu práctica de figuras: *toda* figura tiene área y perímetro, pero no existe una fórmula para «una figura» en general; cada figura concreta tiene la suya.

Para eso existen las **clases abstractas**. Así es tu clase padre:

FigurasGeometricas.java

```java
public abstract class FigurasGeometricas {          // abstract: no se puede hacer new
    protected String nombre, color;
    int grosorBorde;

    public FigurasGeometricas(String nombre, String color, int grosorBorde) {
        this.nombre = nombre;
        this.color = color;
        this.grosorBorde = grosorBorde;
    }

    public abstract void dibujar();                 // sin cuerpo: solo dice QUÉ
    public abstract void area();
    public abstract void perimetro();
}
```

Un **método abstracto** no tiene cuerpo (no lleva llaves y termina en `;`): solo dice «toda figura **debe** saber calcular su área». Cada hija está obligada a escribir el cómo:

Circulo.java

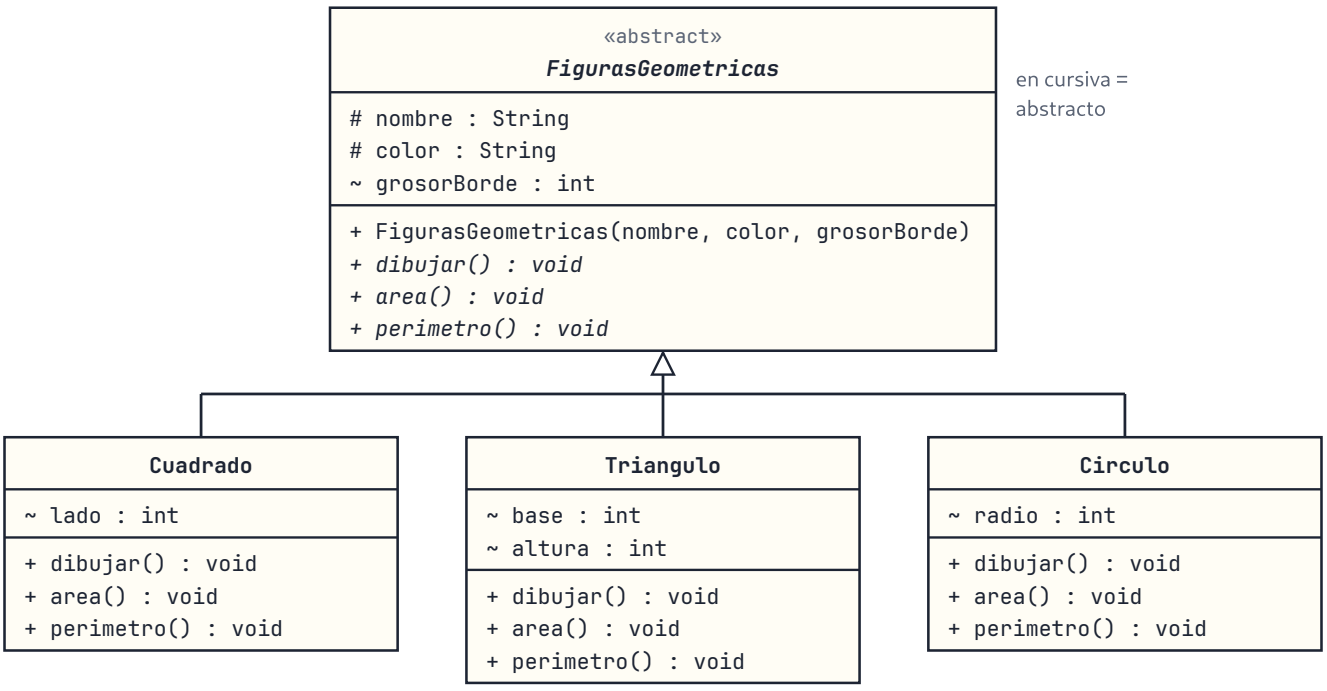
```java
public class Circulo extends FigurasGeometricas {
    int radio;

    public Circulo(int radio, String nombre, String color, int grosorBorde) {
        super(nombre, color, grosorBorde);
        this.radio = radio;
    }

    @Override
    public void dibujar() {
        System.out.println(" *** ");
        System.out.println("*****");
        System.out.println(" *** ");
    }

    @Override
    public void area() {                              // aquí sí dice CÓMO
        System.out.println("area = " + (3.1416 * radio * radio));
    }

    @Override
    public void perimetro() {
        System.out.println("perimetro = " + (2 * 3.1416 * radio));
    }
}
```



💡 **REGLAS DE LAS CLASES ABSTRACTAS**

1. **No se pueden crear objetos** de ellas: `new FigurasGeometricas(...)` da error. (¿Cómo dibujarías «una figura» sin saber cuál?)
2. Pueden tener atributos, constructor y métodos normales (con cuerpo), además de los abstractos.
3. Si una clase tiene **al menos un** método abstracto, la clase **debe** declararse `abstract`.
4. Una hija **debe** sobrescribir **todos** los métodos abstractos del padre (o ser abstracta también); si le falta uno, no compila.

En UML, lo abstracto se escribe en *cursiva* o con la etiqueta «abstract». Fíjate también en el `~`: `lado`, `radio` y `grosorBorde` no llevan modificador, así que su acceso es de paquete.

6.4 Polimorfismo: un mensaje, muchas formas

Poli = muchas, **morfismo** = formas. La idea tiene dos partes:

1. Una variable del tipo del **padre** puede guardar un objeto de **cualquiera de sus hijas** (porque una Suite **es una** Habitación).
2. Cuando llamas a un método sobrescrito, se ejecuta la versión del **objeto real**, no la del tipo de la variable.

```
Habitacion h1 = new HabitacionSencilla(101, 800.0, 1);   // ✔ una sencilla ES UNA habitación
Habitacion h2 = new Suite(301, 3500.0, true, false);     // ✔ una suite ES UNA habitación

h1.mostrarInfo();      // ejecuta la versión de HabitacionSencilla
h2.mostrarInfo();      // ejecuta la versión de Suite
```

Consola

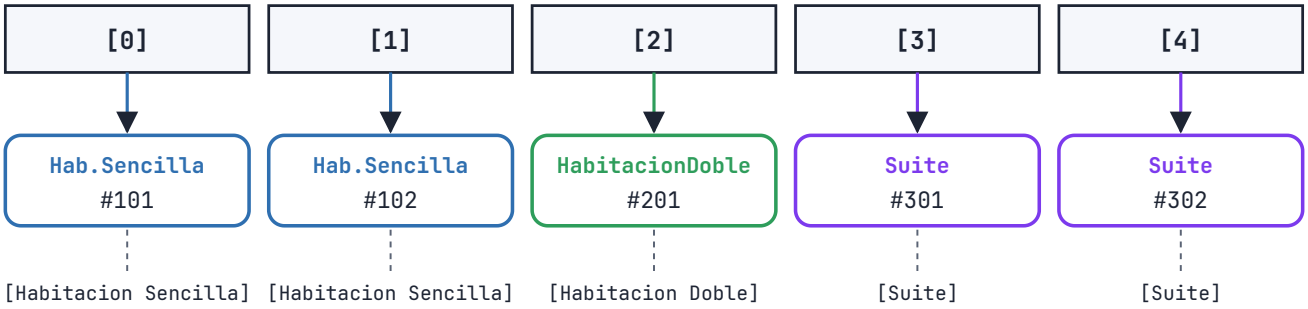
```
[Habitacion Sencilla]
Habitacion #101
  Precio por noche: $800.0
  Estado: Libre
  Camas individuales: 1
[Suite]
Habitacion #301
  Precio por noche: $3500.0
  Estado: Libre
  Jacuzzi: Si
  Vista al mar: No
```

¿Y dónde está lo poderoso? En tu `HotelApp`: un solo arreglo de tipo `Habitacion` guarda sencillas, dobles y suites, y un solo `for` las muestra a todas:

```
Habitacion habitaciones[] = new Habitacion[5];
habitaciones[0] = new HabitacionSencilla(101, 800.0, 1);
habitaciones[1] = new HabitacionSencilla(102, 950.0, 2);
habitaciones[2] = new HabitacionDoble(201, 1500.0, 2);
habitaciones[3] = new Suite(301, 3500.0, true, false);
habitaciones[4] = new Suite(302, 4200.0, true, true);

for (int i = 0; i < habitaciones.length; i++) {
    habitaciones[i].mostrarInfo();     // ¿cuál versión? ¡la del objeto real!
}
```

```
Habitacion habitaciones[] = new Habitacion[5];
```



for (...) habitaciones[i].mostrarInfo(); → cada objeto responde con SU propia versión

El `for` no sabe (ni le importa) de qué tipo es cada habitación: cada objeto responde a su manera. Y si mañana agregas una clase `Penthouse`, el `for` no cambia ni una letra. Lo mismo pasa con tus figuras:

```java
FigurasGeometricas figuras[] = new FigurasGeometricas[3];
figuras[0] = new Cuadrado(3, "Cuadrado", "Rosa", 1);
figuras[1] = new Triangulo(4, 3, "Triangulo", "Azul", 1);
figuras[2] = new Circulo(2, "Circulo", "Verde", 1);

for (int i = 0; i < figuras.length; i++) {
    figuras[i].area();          // cada figura calcula SU área
}
```

Consola

```
area = 9
area = 6.0
area = 12.5664
```

⚠️ **CUIDADO**

Con una variable del tipo del padre **solo puedes llamar a métodos que existan en el padre**. Si `Suite` tuviera un método `prenderJacuzzi()`, `h2.prenderJacuzzi()` daría error, porque `h2` es de tipo `Habitacion`, y una habitación cualquiera no tiene jacuzzi.

6.5 Los 4 pilares de la POO

Tu nota del 25 de agosto los llama «propiedades de los objetos». Ya los viste todos:

Abstracción

Representar solo lo esencial de algo y definir **qué** hace, sin decir **cómo**.

En tu código: `FigurasGeometricas` con `area()` abstracto.

Encapsulamiento

Ocultar los datos (`private`) y permitir usarlos solo mediante métodos que los protegen.

En tu código: `setEdad()` , que rechaza edades negativas.

Herencia

Una clase hija reutiliza los atributos y métodos de una clase padre (`extends`).

En tu código: `Futbolista extends Persona` .

Polimorfismo

Un mismo método se comporta distinto según el objeto que lo ejecuta.

En tu código: `mostrarInfo()` en Sencilla, Doble y Suite.

🎯 **PARA EL EXAMEN**

«Menciona y explica los 4 pilares de la POO con un ejemplo» es de las preguntas más comunes. Apréndete los cuatro recuadros de arriba: definición corta + ejemplo de tu propio código.

En resumen

- **Sobrescritura:** la hija redefine un método heredado (mismos parámetros, `@Override`).
- **Sobrecarga:** mismo nombre, parámetros distintos, en la misma clase.
- Una clase `abstract` no se puede instanciar; sus métodos abstractos no tienen cuerpo y las hijas deben implementarlos.
- **Polimorfismo:** una variable del padre guarda hijas, y cada objeto ejecuta **su** versión del método.
- Los 4 pilares: abstracción, encapsulamiento, herencia y polimorfismo.

Prácticas de la Lección 6

Práctica 6.1 · ¿Sobrecarga o sobrescritura? SIN COMPU

Caso	Respuesta
a) En `Calculadora` : `sumar(int, int)` y `sumar(double, double)` .	
b) `Circulo` redefine el `area()` que heredó de `FigurasGeometricas` .	
c) `Usuario()` y `Usuario(String correo, String contraseña)` .	
d) `Suite` escribe su propio `mostrarInfo()` con `@Override` .	
e) En la clase `Persona` : `saludar()` y `saludar(String nombre)` .	

Práctica 6.2 · ¿Qué imprime? SIN COMPU

```
class Animal {
```

```
    public void hablar() {
        System.out.println("...");
    }
}
class Perro extends Animal {
    @Override
    public void hablar() {
        System.out.println("Guau");
    }
}
class Gato extends Animal {
    @Override
    public void hablar() {
        System.out.println("Miau");
    }
}
class Pollito extends Animal {
}

public class Main {
    public static void main(String[] args) {
        Animal zoo[] = new Animal[4];
        zoo[0] = new Perro();
        zoo[1] = new Gato();
        zoo[2] = new Pollito();
        zoo[3] = new Gato();
        for (int i = 0; i < zoo.length; i++) {
            zoo[i].hablar();
        }
    }
}
```

Práctica 6.3 · ¿Compila o no? SIN COMPU

Marca cada línea de `a)` a `e)` con ✔ o ✘ y explica por qué.

```java
abstract class Instrumento {
    public abstract void tocar();
}

class Guitarra extends Instrumento {
    @Override
    public void tocar() {
        System.out.println("Rasgueo");
    }
}

class Tambor extends Instrumento {          // a)
}

public class Main {
    public static void main(String[] args) {
        Instrumento i1 = new Instrumento();  // b)
        Instrumento i2 = new Guitarra();     // c)
        Guitarra g = new Instrumento();      // d)
        i2.tocar();                          // e)
    }
}
```

Práctica 6.4 · Empleados A PROGRAMAR

Crea la clase abstracta `Empleado` con el atributo `nombre`, el método abstracto `double calcularSueldo()` y el método normal `mostrar()`, que imprime «*nombre* gana $*sueldo*» usando `calcularSueldo()`. Crea dos hijas: `EmpleadoFijo` (con `sueldoMensual`) y `EmpleadoPorHoras` (con `horas` y `pagoPorHora`; su sueldo es horas × pago). En `main`, guarda tres empleados en un arreglo `Empleado[]` y recórrelo con un `for` llamando a `mostrar()`.

Autoevaluación: ¿ya puedes…?

- ☐ Llenar de memoria la tabla de sobrecarga contra sobrescritura.
- ☐ Explicar para qué sirve `@Override`.
- ☐ Decir las 4 reglas de las clases abstractas.
- ☐ Explicar el polimorfismo con el ejemplo del hotel o de las figuras.
- ☐ Definir los 4 pilares de la POO con un ejemplo cada uno.

LECCIÓN 7 · DÍA 7

7 Arreglos, ArrayList y excepciones

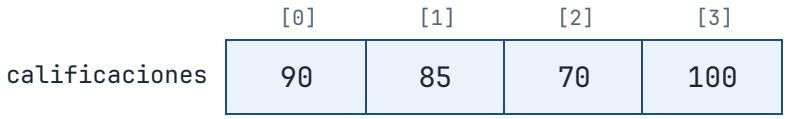
Guardar muchos objetos juntos y evitar que el programa truene

Vas a aprender:

- Crear y recorrer arreglos
- Arreglos de objetos y el famoso `null`
- Buscar dentro de un arreglo
- `ArrayList` : la lista que crece sola
- El for-each y el método `toString()`
- Manejar errores con `try` / `catch`

7.1 Arreglos: muchas cajitas en fila

Un **arreglo** es una fila de cajitas del **mismo tipo**, todas bajo un mismo nombre. Cada cajita se identifica con un número llamado **índice**, que **empieza en 0**:



Tres formas de crear un arreglo (se usa una)

```
int calificaciones[] = new int[4];          // 4 cajitas, todas en 0 (así está en tus proyectos)
int[] calificaciones = new int[4];          // exactamente lo mismo: los [] pueden ir en el tipo
int calificaciones[] = {90, 85, 70, 100};   // crearlo y llenarlo de una vez
```

Usarlo

```
calificaciones[0] = 95;                     // cambiar la primera cajita
System.out.println(calificaciones[3]);      // leer la última: 100
System.out.println(calificaciones.length); // cuántas cajitas hay: 4
```

⚠️ CUIDADO: EL ÚLTIMO ÍNDICE ES LENGTH − 1

Un arreglo de 4 elementos tiene índices del 0 al 3. Pedir `calificaciones[4]` hace que el programa truene con `ArrayIndexOutOfBoundsException`. Por eso los `for` que recorren arreglos usan `i < arreglo.length` (menor que, **no** menor o igual).

```
int calificaciones[] = {90, 85, 70, 100};

double suma = 0;                                    // double para no perder decimales
for (int i = 0; i < calificaciones.length; i++) {   // i va de 0 a 3
    suma = suma + calificaciones[i];
}
double promedio = suma / calificaciones.length;
System.out.println("Promedio: " + promedio);
```

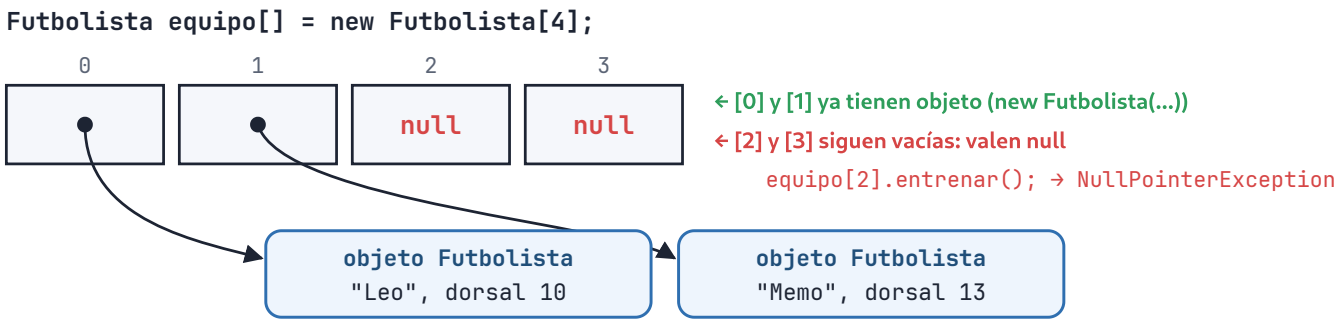
Consola

```
Promedio: 86.25
```

¿Por qué `suma` es `double`? Si fuera `int`, `345 / 4` sería división entera y daría `86` en lugar de `86.25`. ¡La trampa de la Lección 1 sigue viva!

7.2 Arreglos de objetos

Tu propio comentario en el proyecto de futbolistas lo dice perfecto: *esto solo reserva los espacios; todos valen* `null` *hasta que cada posición se instancie con* `new`.




```
Futbolista equipo[] = new Futbolista[4];                    // 4 lugares VACÍOS (null)
equipo[0] = new Futbolista("Leo", "Messi", 1, 37, 10, "Delantero");  // ahora sí hay objeto en [0]
equipo[0].entrenar();                                        // ✔
equipo[2].entrenar();                                        // ✘ NullPointerException
```

`null` significa «aquí no hay ningún objeto». Llamar un método sobre `null` provoca `NullPointerException`. Por eso en tus menús revisabas `EquipoFutbol[jugador] == null` antes de usar al jugador.

Buscar en un arreglo

Este es el patrón de tu hotel para encontrar una habitación por su número: empezar con `posicion = -1` («todavía no la encuentro») y recorrer todo el arreglo:

```
int numero = 201;                // la habitación que buscamos
int posicion = -1;               // -1 significa "todavía no la encuentro"

for (int i = 0; i < habitaciones.length; i++) {
    if (habitaciones[i].getNumero() == numero) {
        posicion = i;            // ¡la encontré! guardo dónde está
    }
}

if (posicion == -1) {
    System.out.println("No existe la habitación #" + numero);
} else {
    System.out.println("La habitación #" + numero + " está en la posición " + posicion);
    habitaciones[posicion].reservar();
}
```

Consola

```
La habitación #201 está en la posición 2
La habitacion #201 quedo reservada
```

7.3 ArrayList: la lista que crece sola

El problema de los arreglos es que su tamaño es **fijo**: si creas uno de 5, no le cabe un sexto. Un `ArrayList` es una lista que **crece y se encoge sola** conforme agregas o quitas elementos. Necesita `import java.util.ArrayList;`.

```
ArrayList<String> tareas = new ArrayList<>();   // empieza vacía

tareas.add("Estudiar Java");                    // posición 0
tareas.add("Hacer la práctica");                // posición 1
tareas.add("Dormir temprano");                  // posición 2

System.out.println(tareas.size());              // cuántas hay
System.out.println(tareas.get(0));              // la de la posición 0
tareas.set(2, "Repasar herencia");              // reemplaza la posición 2
tareas.remove(1);                               // borra la posición 1 (las demás se recorren)
System.out.println(tareas);                     // imprime toda la lista
```

Consola

```
3
Estudiar Java
[Estudiar Java, Repasar herencia]
```

Método	Qué hace
`lista.add(x)`	Agrega `x` al final
`lista.add(i, x)`	Inserta `x` en la posición `i` (recorre las demás)
`lista.get(i)`	Regresa el elemento de la posición `i`
`lista.set(i, x)`	Reemplaza el elemento de la posición `i`
`lista.remove(i)`	Borra el elemento de la posición `i`
`lista.size()`	Cuántos elementos tiene
`lista.isEmpty()`	¿Está vacía?
`lista.contains(x)`	¿Contiene a `x`?

💡 LO QUE VA ENTRE < >

`ArrayList<String>` dice qué tipo de cosas guarda la lista. Con tus propias clases: `ArrayList<Usuario> listaUsuarios = new ArrayList<>();`. Dos detalles: para números se usan las clases envoltorio `Integer` y `Double` (`ArrayList<Integer>`, no `ArrayList<int>`), y aunque se puede crear sin los `<>` (como en tu ejercicio de tareas), es mejor indicar siempre el tipo.

Recorrer con for-each

Para recorrer toda una lista (o un arreglo) sin manejar índices existe el **for-each**. Se lee «**para cada** Usuario `x` **de** la lista». Así revisabas si un correo ya estaba registrado:

```
for (Usuario x : listaUsuarios) {
    if (x.getCorreo().equalsIgnoreCase(usuario)) {
        repetido = true;
    }
}
```

	Arreglo	ArrayList
Tamaño	Fijo	Crece y se encoge
Crear	`new Usuario[5]`	`new ArrayList<>()`
Leer	`arr[i]`	`lista.get(i)`
Cambiar	`arr[i] = x`	`lista.set(i, x)`
¿Cuántos hay?	`arr.length`	`lista.size()`
Tipos	Primitivos y objetos	Solo objetos (`Integer` , no `int`)

⚠️ **TRES «TAMAÑOS» DISTINTOS: NO LOS CONFUNDAS**

Arreglo: `arr.length` (sin paréntesis). Texto: `texto.length()` (con paréntesis). ArrayList: `lista.size()` .

7.4 Imprimir objetos: toString()

En tu ejercicio de la lista de usuarios, la opción «Mostrar lista» imprimía algo como esto:

Consola

```
[com.mycompany.e_arraylistclase.Usuario@1b6d3586]
```

Java no sabe cómo quieres que se vea tu objeto, así que usa el `toString()` que todas las clases heredan de `Object` : el nombre de la clase, una `@` y un código. La solución es **sobrescribir** `toString()` (¡herencia y sobrescritura juntas!):

Usuario.java

```
@Override
public String toString() {              // así quiero que se vea mi objeto como texto
    return "Usuario: " + correo;
}
```

```
ArrayList<Usuario> listaUsuarios = new ArrayList<>();
listaUsuarios.add(new Usuario("jorge@gmail.com", "Clave#2026"));
listaUsuarios.add(new Usuario("ana@gmail.com", "Gatos#123"));
System.out.println(listaUsuarios);                // ahora usa TU toString()

for (Usuario x : listaUsuarios) {                 // for-each: "para cada Usuario x de la lista"
    System.out.println("Correo: " + x.getCorreo());
}
```

Consola

```
[Usuario: jorge@gmail.com, Usuario: ana@gmail.com]
Correo: jorge@gmail.com
Correo: ana@gmail.com
```

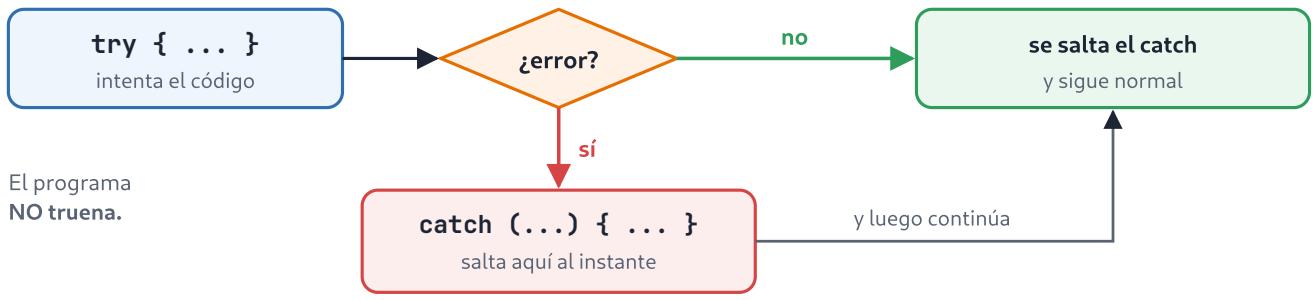
7.5 Excepciones: cuando algo sale mal

Una **excepción** es un error que ocurre **mientras el programa se ejecuta** (no al compilar). Si nadie la atiende, el programa «truena»: se detiene y muestra un mensaje en rojo. Las más comunes:

Excepción	Cuándo pasa	Ejemplo
`InputMismatchException`	`nextInt()` recibe algo que no es número	Escriben «hola» en vez de 5
`NumberFormatException`	Convertir a número un texto que no lo es	`Integer.parseInt("hola")`
`ArithmeticException`	Dividir un `int` entre cero	`10 / 0`
`ArrayIndexOutOfBoundsException`	Índice fuera del arreglo	`arr[3]` cuando `length` es 3
`IndexOutOfBoundsException`	Posición fuera de un ArrayList	`lista.get(0)` con la lista vacía
`NullPointerException`	Usar un objeto que es `null`	`equipo[2].entrenar()` sin haber hecho `new`

try / catch

```
try {
    // código que podría fallar
} catch (TipoDeExcepcion e) {
    // qué hacer si falla
}
```



Si nada falla dentro del `try`, el `catch` se ignora. Si algo falla, Java **salta al instante** al `catch` (lo que quedaba del `try` ya no se ejecuta) y después el programa **continúa** normalmente. Este es el patrón de tu práctica de figuras para pedir un número hasta que sea válido:

```java
int lado = 0;
boolean valido;
do {
    System.out.println("Dame la medida del lado:");
    try {
        lado = sn.nextInt();          // si escriben "hola", aquí truena...
        valido = true;                // (esta línea ya no se ejecuta)
    } catch (InputMismatchException e) {
        System.out.println("Error: debes teclear un número");   // ...y salta aquí
        sn.nextLine();                // limpia lo que escribieron mal
        valido = false;
    }
} while (!valido);                    // repite mientras NO sea válido
System.out.println("Lado = " + lado);
```

Consola

```
Dame la medida del lado:
hola
Error: debes teclear un número
Dame la medida del lado:
5
Lado = 5
```

> ⚠️ **¿POR QUÉ EL SN.NEXTLINE() DENTRO DEL CATCH?**
>
> Cuando `nextInt()` falla, el texto mal escrito («hola») **se queda** esperando en el teclado. Si no lo limpias, la siguiente vuelta vuelve a leer «hola», vuelve a fallar… y el ciclo se repite para siempre.

Lanzar tus propias excepciones: throw

También puedes provocar un error a propósito con `throw` cuando un dato no cumple tus reglas, como en tu validación de contraseñas:

```java
try {
    if (contraseña.length() < 8) {
        throw new Exception("La contraseña debe tener mínimo 8 caracteres");   // lanza el error
    }
    System.out.println("Contraseña aceptada");
} catch (Exception e) {
    System.out.println(e.getMessage());     // muestra el mensaje que pusimos arriba
}
```

Consola (con la contraseña gato123)

```
La contraseña debe tener mínimo 8 caracteres
```

`e.getMessage()` regresa el mensaje que pusiste al crear la excepción. Existe además un bloque opcional `finally { ... }`, que se ejecuta **siempre**, haya error o no.

En resumen

- Un arreglo tiene tamaño fijo e índices de `0` a `length - 1`.
- Un arreglo de objetos empieza lleno de `null`: cada posición necesita su `new`.
- `ArrayList` crece sola: `add`, `get`, `set`, `remove`, `size`.
- El for-each recorre todo sin índices: `for (Tipo x : lista)`.
- Sobrescribe `toString()` para imprimir tus objetos de forma legible.
- `try` / `catch` atrapa los errores para que el programa no truene.

Prácticas de la Lección 7

Práctica 7.1 · ¿Qué imprime? SIN COMPU

```java
int n[] = {4, 8, 15, 16, 23};
System.out.println(n.length);
System.out.println(n[1] + n[3]);
n[0] = n[4] - n[2];
System.out.println(n[0]);
int suma = 0;
for (int i = 0; i < n.length; i = i + 2) {
    suma = suma + n[i];
}
System.out.println(suma);
```

Práctica 7.2 · ¿Qué excepción lanza? SIN COMPU

Escribe el nombre de la excepción (o «ninguna»). Una es trampa.

```java
// a)
int edades[] = {18, 20, 19};
System.out.println(edades[3]);
// b)
int numero = Integer.parseInt("12a");
// c)
String nombres[] = new String[2];
System.out.println(nombres[0].length());
// d)
int total = 100, personas = 0;
System.out.println(total / personas);
// e)
double cuenta = 100, amigos = 0;
System.out.println(cuenta / amigos);
```

Práctica 7.3 · ¿Qué imprime? SIN COMPU

```
ArrayList<String> letras = new ArrayList<>();
letras.add("A");
letras.add("B");
letras.add("C");
letras.add(1, "X");  // inserta en la posición 1
letras.remove(2);
System.out.println(letras + " " + letras.size());
System.out.println(letras.contains("B"));
```

Práctica 7.4 · Calificaciones del grupo A PROGRAMAR

Crea la clase `Alumno` (`nombre`, `calificacion`, getters y un `toString()`). En `main`, usa un `ArrayList<Alumno>` y un menú con: *1. Agregar alumno · 2. Mostrar todos · 3. Buscar por nombre · 4. Promedio del grupo · 5. Salir*. Usa `try` / `catch` para que el programa no truene si escriben letras donde va un número.

Autoevaluación: ¿ya puedes…?

- ☐ Crear un arreglo, llenarlo y recorrerlo con `for` sin salirte de los índices.
- ☐ Explicar por qué un arreglo de objetos recién creado está lleno de `null`.
- ☐ Usar `ArrayList` con `add`, `get`, `set`, `remove` y `size`.
- ☐ Recorrer una lista con for-each y sobrescribir `toString()`.
- ☐ Escribir un `try` / `catch` que pida un número hasta que sea válido.

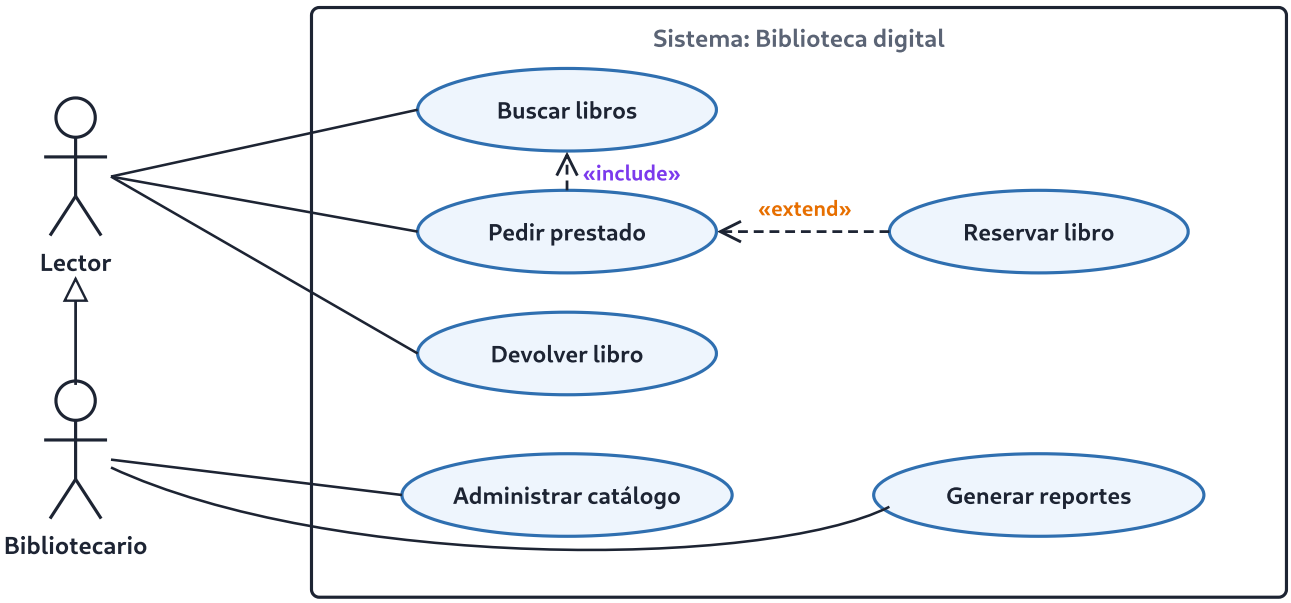
Anexo: diagramas de casos de uso

Además del diagrama de clases, en clase vieron el **diagrama de casos de uso** (también es UML). Como escribieron en su tarea, su propósito es mostrar **qué** hace el sistema desde el punto de vista de quien lo usa, sin entrar en **cómo** lo hace por dentro.

Elemento	Se dibuja como	Qué representa
Actor	Monito de palitos	Alguien o algo externo que usa el sistema (una persona u otro sistema). Es un **rol**, no una persona específica: «Lector», no «Juan».
Caso de uso	Óvalo con un verbo	Una función que ofrece el sistema: «Buscar libros», «Pedir prestado».
Límite del sistema	Rectángulo	Los casos de uso van **dentro**; los actores, **afuera**.
Asociación	Línea continua	Qué actor participa en qué caso de uso.

Las tres relaciones

Relación	Se dibuja como	Significa	Ejemplo
«include»	Flecha punteada hacia el caso incluido	El caso base **siempre** hace también el otro	Pedir prestado **incluye** buscar el libro
«extend»	Flecha punteada hacia el caso base	Comportamiento **opcional**: solo pasa bajo cierta condición	Reservar **extiende** a pedir prestado (solo si el libro ya está prestado)
Generalización	Línea con triángulo hueco hacia el general	Herencia entre actores: el hijo puede hacer todo lo del padre, y más	El bibliotecario **es un** lector con más permisos



El diagrama de la biblioteca digital de su tarea, simplificado.

✅ **TRUCO PARA NO CONFUNDIR INCLUDE Y EXTEND**

include = «siempre, obligatorio»: la flecha sale del caso que **necesita** al otro. **extend** = «a veces, opcional»: la flecha sale del caso **opcional** y apunta al caso que extiende.

Práctica A.1 · Dibuja el diagrama SIN COMPU

Un **cajero automático**: el **cliente** puede consultar su saldo, retirar efectivo y depositar. En esas tres operaciones el sistema **siempre** valida el NIP. Al retirar, **si el cliente lo pide**, se imprime un comprobante. El **técnico** recarga el efectivo del cajero.

Chuleta de repaso

Para repasar la noche antes del examen. Si algo de aquí no te suena, regresa a su lección.

Esqueleto y Scanner

```java
import java.util.Scanner;

public class App {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);
        int n = sn.nextInt();
        sn.nextLine();     // traga el Enter
        String s = sn.nextLine();
        System.out.println(s + " " + n);
    }
}
```

Tipos y operadores

- `int` · `double` · `boolean` · `char` (`'a'`) · `String` (`"a"`)
- `7 / 2` → `3` · `7.0 / 2` → `3.5` · `7 % 2` → `1`
- `== != < > <= >=` · `&&` (y) · `||` (o) · `!` (no)
- `"a" + 1 + 2` → `"a12"` · `1 + 2 + "a"` → `"3a"`
- Textos: `s.equals("x")`, nunca `==`

Decisiones

```java
if (x > 0) { ... }
else if (x < 0) { ... }
else { ... }

switch (op) {
    case 1:
        ...
        break;      // sin break, sigue de largo
    default:
        ...
}
```

Ciclos

```java
for (int i = 0; i < n; i++) { ... }
while (cond) { ... }          // 0 o más veces
do { ... } while (cond);      // 1 o más veces
for (Tipo x : lista) { ... }  // for-each
```

Una clase completa

```java
public class Perro {
    private String nombre;      // atributo

    public Perro(String nombre) { // constructor
        this.nombre = nombre;
    }

    public String getNombre() { // getter
        return nombre;
    }

    public void setNombre(String n) { // setter
        this.nombre = n;
    }

    public void ladrar() {      // método
        System.out.println("Guau");
    }
}

Perro p = new Perro("Rex");     // objeto
p.ladrar();
```

Herencia, abstractas y polimorfismo

```java
public abstract class Animal {
    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
    }

    public abstract void hablar(); // sin cuerpo
}

public class Gato extends Animal {
    public Gato(String nombre) {
        super(nombre);          // 1.ª línea
    }

    @Override
    public void hablar() {
        System.out.println("Miau");
    }
}

Animal a = new Gato("Tom"); // padre guarda hija
a.hablar();                 // Miau
```

Arreglos y ArrayList

```java
int arr[] = new int[5];    // índices 0 a 4
arr[0] = 10;
int n = arr.length;        // 5

ArrayList<String> l = new ArrayList<>();
l.add("a");   l.get(0);   l.set(0, "b");
l.remove(0);  l.size();   l.isEmpty();
```

Excepciones

```java
try {
    int n = sn.nextInt();
} catch (InputMismatchException e) {
    System.out.println("No es número");
    sn.nextLine();                // limpia
}

throw new Exception("mensaje");    // lanzar
```

UML

- `+` public · `-` private · `#` protected · `~` paquete
- Atributo: `- nombre : String`
- Método: `+ area(lado : int) : double`
- Herencia: línea con triángulo hueco hacia el padre
- Abstracto: en *cursiva* o con «abstract»
- Casos de uso: actor, óvalo, rectángulo; «include» = siempre, «extend» = a veces

Palabras clave

- `this`: este objeto · `super(...)`: constructor del padre · `super.m()`: método del padre
- `static`: de la clase, compartido · `final`: no cambia
- `abstract`: sin objetos / sin cuerpo · `@Override`: sobrescribe
- `new`: crea un objeto · `null`: no hay objeto

Los errores que más se repiten

- Falta un `;` o una `}`
- Constructor con `void` (deja de ser constructor)
- Método no `void` sin `return`
- Olvidar `this.` en el constructor
- Comparar textos con `==`
- `i <= arr.length` en vez de `i < arr.length`
- Arreglo de objetos sin `new` en cada posición
- Hija sin `super(...)` cuando el padre no tiene constructor vacío
- Olvidar el `break` en un `case`

Examen de práctica

INSTRUCCIONES

Hazlo como si fuera el real: **sin apuntes, en papel y con 90 minutos**. Al terminar, revísalo con las soluciones del final. Lo que falles te dice qué lección repasar.

I. Teoría

Responde con tus palabras, en dos o tres líneas.

1. ¿Qué es una clase y qué es un objeto? Da un ejemplo de cada uno.

2. ¿Qué es el encapsulamiento y cómo se logra en Java?

3. Explica la diferencia entre sobrecarga y sobrescritura.

4. ¿Qué es una clase abstracta? ¿Se pueden crear objetos de ella?

5. ¿Para qué sirve la palabra `super`? Menciona sus dos usos.

6. ¿Qué significa que un atributo o un método sea `static`?

7. ¿Por qué los `String` se comparan con `equals()` y no con `==`?

8. ¿Qué es un constructor y en qué se diferencia de un método normal?

9. ¿Qué es el polimorfismo? Da un ejemplo.

10. Menciona los 4 pilares de la POO.

II. ¿Qué imprime?

Ejercicio 1.

```java
class Figura {
    protected String nombre;

    public Figura(String nombre) {
        this.nombre = nombre;
        System.out.println("Creando " + nombre);
    }

    public void describir() {
        System.out.println("Soy una figura");
    }
}

class Cuadro extends Figura {
    private int lado;

    public Cuadro(int lado) {
        super("cuadro");
        this.lado = lado;
    }

    @Override
    public void describir() {
        System.out.println("Soy un " + nombre + " de lado " + lado);
    }
}

public class Main {
    public static void main(String[] args) {
        Figura f1 = new Figura("algo");
        Figura f2 = new Cuadro(5);
        f1.describir();
        f2.describir();
    }
}
```

Ejercicio 2.

```
class Boleto {
    private static int vendidos = 0;
    private int numero;

    public Boleto() {
        vendidos++;
        numero = vendidos;
    }

    public int getNumero() {
        return numero;
    }

    public static int getVendidos() {
        return vendidos;
    }
}

public class Main {
    public static void main(String[] args) {
        Boleto b[] = new Boleto[3];
        for (int i = 0; i < b.length; i++) {
            b[i] = new Boleto();
        }
        System.out.println(b[0].getNumero() + " " + b[2].getNumero());
        System.out.println("Vendidos: " + Boleto.getVendidos());

        String r = "";
        for (int i = b.length - 1; i >= 0; i--) {
            r = r + b[i].getNumero();
        }
        System.out.println(r);
    }
}
```

III. Encuentra y corrige los errores

Este código tiene **5 errores**. Cuatro impiden que compile; el otro compila, pero es un error de lógica. Encuéntralos y escribe la corrección.

```
class Libro {
    private String titulo;
    private int paginas;

    public void Libro(String titulo, int paginas) {
        this.titulo = titulo;
        this.paginas = paginas;
    }

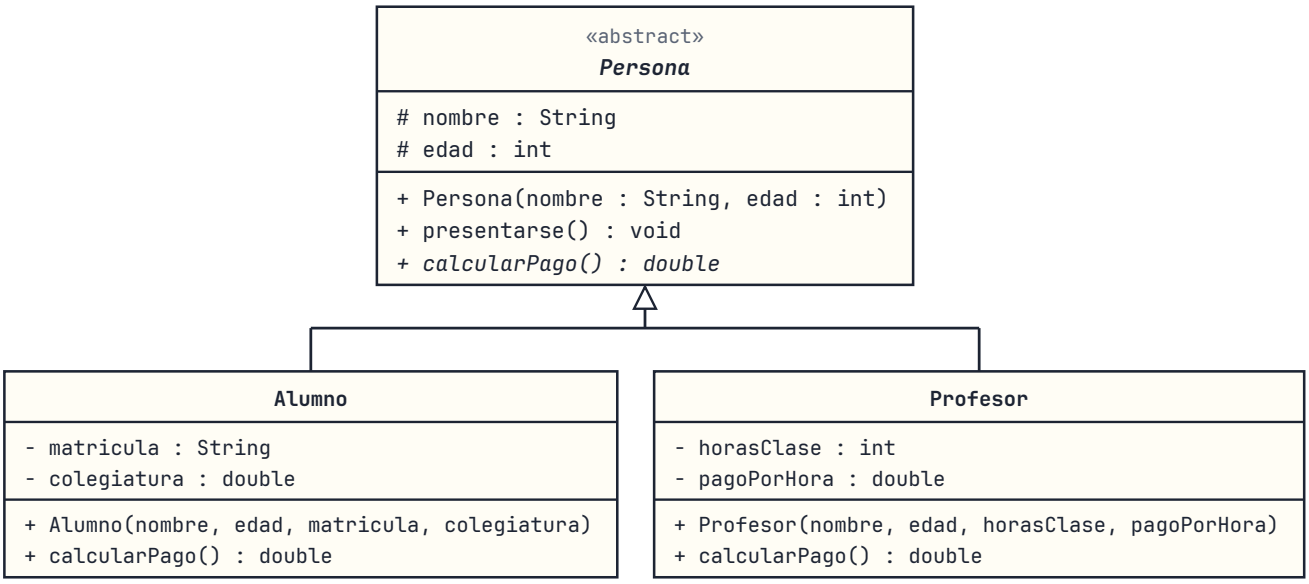
    public String getTitulo() {
        return titulo;
    }

    public int getPaginas() {
        System.out.println(paginas);
    }
}

public class Main {
    public static void main(String[] args) {
        Libro l = new Libro("Java", 300)
        if (l.getTitulo() == "Java") {
            System.out.println("Encontrado");
        }
        l.paginas = 500;
    }
}
```

IV. Del diagrama al código

Escribe las tres clases del diagrama:



- `presentarse()` imprime «Hola, soy *nombre* y tengo *edad* años».
- En `Alumno`, `calcularPago()` regresa la colegiatura; en `Profesor`, regresa `horasClase × pagoPorHora`.
- Después escribe un `main` que guarde un alumno y un profesor en un `ArrayList<Persona>` y la recorra con for-each, llamando a `presentarse()` y mostrando lo que regresa `calcularPago()`.

V. Diagrama de casos de uso

Dibuja el diagrama de una **tienda en línea**: el **cliente** puede buscar productos, agregarlos al carrito y pagar. Pagar **siempre** incluye validar la tarjeta. Al pagar, **si el cliente quiere**, puede aplicar un cupón de descuento. El **administrador** da de alta productos y consulta las ventas.

Soluciones

Revísalas después de intentarlo. En los programas hay muchas soluciones correctas: si el tuyo hace lo mismo, ¡está bien! Todas las salidas que ves aquí son reales: cada programa se compiló y se ejecutó.

LECCIÓN 1

1.1 ¿Qué imprime?

Consola

```
7
Resultado: 52
Resultado: 7
2
2.5
```

Trampa 1 (línea 2): en cuanto aparece el texto `"Resultado: "`, el `+` pega en lugar de sumar: pega el 5 y luego el 2. **Trampa 2** (línea 4): `5 / 2` entre dos `int` es división entera.

1.2 Los 6 errores

1. `public class programa` → `public class Programa`: debe llamarse igual que el archivo, con mayúscula.
2. `int edad = 19` → falta el `;` al final.
3. `'Ana'` → `"Ana"`: los textos van entre comillas dobles (las simples son para un solo `char`).
4. `system.out.println` → `System.out.println`, con S mayúscula.
5. `Int anio` → `int anio`, con minúscula.
6. Falta la `}` final que cierra la clase.

1.3 Preséntate

Presentacion.java

```java
import java.util.Scanner;

public class Presentacion {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);

        System.out.println("¿Cómo te llamas?");
        String nombre = sn.nextLine();

        System.out.println("¿Cuántos años tienes?");
        int edad = sn.nextInt();
        sn.nextLine();                      // traga el Enter que dejó nextInt

        System.out.println("¿En qué ciudad vives?");
        String ciudad = sn.nextLine();

        System.out.println("Hola " + nombre + ", tienes " + edad + " años, vives en "
                + ciudad + " y el próximo año tendrás " + (edad + 1) + ".");
    }
}
```

Consola

```
¿Cómo te llamas?
Jorge
¿Cuántos años tienes?
19
¿En qué ciudad vives?
Guadalajara
Hola Jorge, tienes 19 años, vives en Guadalajara y el próximo año tendrás 20.
```

1.4 Ticket con IVA

Ticket.java

```java
import java.util.Scanner;

public class Ticket {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);

        System.out.println("Precio del producto:");
        double precio = sn.nextDouble();
        System.out.println("Cantidad:");
        int cantidad = sn.nextInt();

        double subtotal = precio * cantidad;
        double iva = subtotal * 0.16;
        double total = subtotal + iva;

        System.out.println("Subtotal: $" + subtotal);
        System.out.println("IVA (16%): $" + iva);
        System.out.println("Total: $" + total);
    }
}
```

Consola

```
Precio del producto:
500
Cantidad:
2
Subtotal: $1000.0
IVA (16%): $160.0
Total: $1160.0
```

LECCIÓN 2

2.1 ¿Qué imprime cada uno?

a) `0 1 2` — `i` vale 0, 1 y 2; con 3, la condición `3 < 3` ya es false.
b) `10 7 4 1` — `n` baja de 3 en 3: 10, 7, 4, 1 y −2 (que ya no cumple `n > 0`).
c) `32` — `k` vale 2, 4, 8 y 16 (todavía menor que 20) y luego 32 (ya no): el ciclo para con 32.
d) `1`, `12` y `123`, cada uno en su línea: la fila `f` imprime los números del 1 hasta `f`.

2.2 El switch tramposo

Consola

```
B
C
```

Entra en `case 'b'`, imprime B y, como no hay `break`, sigue al `case 'c'` e imprime C. Ahí sí hay `break`, así que el `default` ya no se ejecuta.

2.3 Par o impar

ParImpar.java

```java
import java.util.Scanner;

public class ParImpar {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);
        System.out.println("Dame un número entero:");
        int numero = sn.nextInt();

        if (numero % 2 == 0) {              // si el residuo entre 2 es 0, es par
            System.out.println(numero + " es par");
        } else {
            System.out.println(numero + " es impar");
        }

        if (numero > 0) {
            System.out.println("y es positivo");
        } else if (numero < 0) {
            System.out.println("y es negativo");
        } else {
            System.out.println("y es cero");
        }
    }
}
```

2.4 Tabla de multiplicar

Tabla.java

```java
import java.util.Scanner;

public class Tabla {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);
        System.out.println("¿Qué tabla quieres?");
        int n = sn.nextInt();

        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = " + (n * i));
        }
    }
}
```

2.5 Calculadora con menú

Calculadora.java

```java
import java.util.Scanner;

public class Calculadora {
```

```
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);
        int opcion;
        double a, b;

        do {
            System.out.println("1. Sumar  2. Restar  3. Multiplicar  4. Dividir  5. Salir");
            opcion = sn.nextInt();

            if (opcion >= 1 && opcion <= 4) {          // solo pide números si va a operar
                System.out.println("Primer número:");
                a = sn.nextDouble();
                System.out.println("Segundo número:");
                b = sn.nextDouble();

                switch (opcion) {
                    case 1:
                        System.out.println("Resultado: " + (a + b));
                        break;
                    case 2:
                        System.out.println("Resultado: " + (a - b));
                        break;
                    case 3:
                        System.out.println("Resultado: " + (a * b));
                        break;
                    case 4:
                        if (b == 0) {
                            System.out.println("No se puede dividir entre cero");
                        } else {
                            System.out.println("Resultado: " + (a / b));
                        }
                        break;
                }
            } else if (opcion != 5) {
                System.out.println("Opción inválida");
            }
        } while (opcion != 5);

        System.out.println("¡Hasta luego!");
    }
}
```

2.6 Del pseudocódigo a Java

Login.java

```
import java.util.Scanner;

public class Login {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);
        String idCorrecto = "admin";                        // idCorrecto ← "admin"
        String passCorrecta = "1234";
        int intentos = 0;
        boolean acceso = false;

        while (intentos < 3 && acceso == false) {           // MIENTRAS ... Y ... HACER
            System.out.println("Id:");
            String id = sn.nextLine();                      // LEER id
            System.out.println("Contraseña:");
            String pass = sn.nextLine();

            if (id.equals(idCorrecto) && pass.equals(passCorrecta)) {   // equals, ¡no ==!
                System.out.println("Bienvenido");
                acceso = true;
            } else {
```

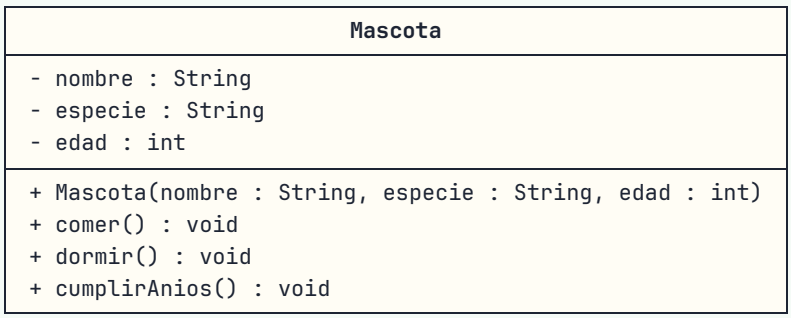
```
                intentos = intentos + 1;
                System.out.println("Datos incorrectos");
            }
        }                                                   // FIN MIENTRAS

        if (acceso == false) {
            System.out.println("Acceso bloqueado");
        }
    }
}
```

También se vale escribir `while (intentos < 3 && !acceso)` e `if (!acceso)`: significan lo mismo.

LECCIÓN 3

3.1 Diagrama de Mascota



Atributos = los sustantivos del enunciado (nombre, especie, edad). Métodos = los verbos (comer, dormir, cumplir años). Agregar un `mostrarDatos()` es un extra útil.

3.2 Programa tu Mascota

Mascota.java

```
public class Mascota {
    // Atributos
    private String nombre;
    private String especie;
    private int edad;

    // Constructor
    public Mascota(String nombre, String especie, int edad) {
        this.nombre = nombre;
        this.especie = especie;
        this.edad = edad;
    }

    // Métodos
    public void comer() {
        System.out.println(nombre + " está comiendo");
    }

    public void dormir() {
        System.out.println(nombre + " está durmiendo... zzz");
    }

    public void cumplirAnios() {
        edad++;
        System.out.println("¡" + nombre + " cumplió " + edad + " años!");
    }

    public void mostrarDatos() {
        System.out.println(nombre + " (" + especie + "), " + edad + " años");
    }
```

```
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Mascota m1 = new Mascota("Firulais", "perro", 3);
        Mascota m2 = new Mascota("Michi", "gato", 1);

        m1.comer();
        m2.dormir();
        m2.cumplirAnios();
        m1.mostrarDatos();
        m2.mostrarDatos();
    }
}
```

Consola

```
Firulais está comiendo
Michi está durmiendo... zzz
¡Michi cumplió 2 años!
Firulais (perro), 3 años
Michi (gato), 2 años
```

3.3 ¿Qué imprime?

Consola

```
Rojo: true
Azul: false
```

Los dos focos empiezan con `encendido = false` (el valor por defecto de un `boolean`). `f1` se presiona una vez: `true`. `f2` se presiona dos veces: false → true → false. Cada objeto guarda su propio estado.

3.4 Los 4 errores

1. `public void Celular(...)`: con `void` deja de ser constructor y se vuelve un método normal. Por eso `new Celular("Samsung", 80)` falla: no hay un constructor con esos parámetros. Corrección: `public Celular(String marca, int bateria)`.
2. `getBateria()` dice que regresa un `int`, pero no tiene `return`. Corrección: `return bateria;`.
3. `bateria = 100`: falta el `;`.
4. `c.apagar()`: la clase `Celular` no tiene ningún método `apagar()`.

3.5 Del diagrama al código

Producto.java

```java
public class Producto {
    private String nombre;
    private double precio;
    private int existencias;

    public Producto(String nombre, double precio, int existencias) {
        this.nombre = nombre;
        this.precio = precio;
        this.existencias = existencias;
    }
```

```
    public void vender(int cantidad) {
        if (cantidad <= existencias) {
            existencias = existencias - cantidad;
            System.out.println("Vendiste " + cantidad + " " + nombre);
        } else {
            System.out.println("No hay suficientes " + nombre);
        }
    }

    public double calcularValorInventario() {
        return precio * existencias;
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        Producto p = new Producto("cuadernos", 35.5, 10);
        p.vender(4);
        p.vender(20);
        System.out.println("Valor del inventario: $" + p.calcularValorInventario());
    }
}
```

Consola

```
Vendiste 4 cuadernos
No hay suficientes cuadernos
Valor del inventario: $213.0
```

LECCIÓN 4

4.1 Cuenta bancaria

CuentaBancaria.java

```
public class CuentaBancaria {
    private String titular;
    private double saldo;

    public CuentaBancaria(String titular, double saldoInicial) {
        this.titular = titular;
        if (saldoInicial >= 0) {
            this.saldo = saldoInicial;
        } else {
            this.saldo = 0;
        }
    }

    public void depositar(double monto) {
        if (monto > 0) {
            saldo = saldo + monto;
            System.out.println("Depósito de $" + monto + ". Saldo: $" + saldo);
        } else {
            System.out.println("El monto debe ser mayor a 0");
        }
    }

    public void retirar(double monto) {
        if (monto <= 0) {
            System.out.println("El monto debe ser mayor a 0");
        } else if (monto > saldo) {
            System.out.println("Fondos insuficientes");
        } else {
```

```
            saldo = saldo - monto;
            System.out.println("Retiro de $" + monto + ". Saldo: $" + saldo);
        }
    }

    public double getSaldo() {          // solo getter: el saldo NO tiene setter
        return saldo;
    }

    public String getTitular() {
        return titular;
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        CuentaBancaria cuenta = new CuentaBancaria("Jorge", 500);
        cuenta.depositar(200);
        cuenta.retirar(1000);
        cuenta.retirar(300);
        cuenta.depositar(-50);
        System.out.println(cuenta.getTitular() + " tiene $" + cuenta.getSaldo());
    }
}
```

Consola

```
Depósito de $200.0. Saldo: $700.0
Fondos insuficientes
Retiro de $300.0. Saldo: $400.0
El monto debe ser mayor a 0
Jorge tiene $400.0
```

¿Por qué no hay `setSaldo()`? Porque permitiría poner cualquier saldo (incluso negativo) y saltarse las reglas de `depositar()` y `retirar()`. El saldo solo debe cambiar con operaciones válidas: eso es encapsulamiento.

4.2 ¿Compila o no?

a) ✘ `codigo` es `private`: solo `Caja` lo puede usar.
b) ✔ `color` es `public`.
c) ✔ `protected` permite el acceso desde el mismo paquete.
d) ✔ Sin modificador = acceso de paquete, y las dos clases están en el mismo.
e) ✘ `abrir()` es `private`.
f) ✔ `cerrar()` es `public`.

4.3 ¿Qué imprime?

Consola

```
1 3
```

`propio` es de cada objeto: cada uno lo sube de 0 a 1, así que `c3.propio` vale 1. `compartido` es `static`: hay uno solo y lo suben los tres constructores, así que vale 3.

4.4 Mejora tu Mascota

Mascota.java

```java
public class Mascota {
    private String nombre;
    private String especie;
    private int edad;
    private static int totalMascotas = 0;      // una sola copia para todas

    public Mascota(String nombre, String especie, int edad) {
        this.nombre = nombre;
        this.especie = especie;
        setEdad(edad);                         // reutiliza la validación del setter
        totalMascotas++;                       // cada mascota nueva suma 1
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getEspecie() {
        return especie;
    }

    public void setEspecie(String especie) {
        this.especie = especie;
    }

    public int getEdad() {
        return edad;
    }

    public void setEdad(int edad) {
        if (edad >= 0) {
            this.edad = edad;
        } else {
            System.out.println("Edad inválida");
        }
    }

    public static int getTotalMascotas() {
        return totalMascotas;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Mascota m1 = new Mascota("Firulais", "perro", 3);
        Mascota m2 = new Mascota("Michi", "gato", -2);
        System.out.println(m2.getNombre() + " tiene " + m2.getEdad() + " años");
        System.out.println("Mascotas creadas: " + Mascota.getTotalMascotas());
    }
}
```

Consola

```
Edad inválida
Michi tiene 0 años
Mascotas creadas: 2
```

LECCIÓN 5

5.1 ¿Herencia o no?

Pareja	Respuesta
Perro – Animal	Un perro **es un** animal → herencia; el padre es `Animal`
Auto – Motor	Un auto **tiene un** motor → atributo
Gerente – Empleado	Un gerente **es un** empleado → herencia; el padre es `Empleado`
Celular – Batería	Un celular **tiene una** batería → atributo
Moto – Vehículo	Una moto **es un** vehículo → herencia; el padre es `Vehiculo`
Escuela – Salón	Una escuela **tiene** salones → atributo (por ejemplo, un `ArrayList<Salon>`)

5.2 ¿Qué imprime?

Consola

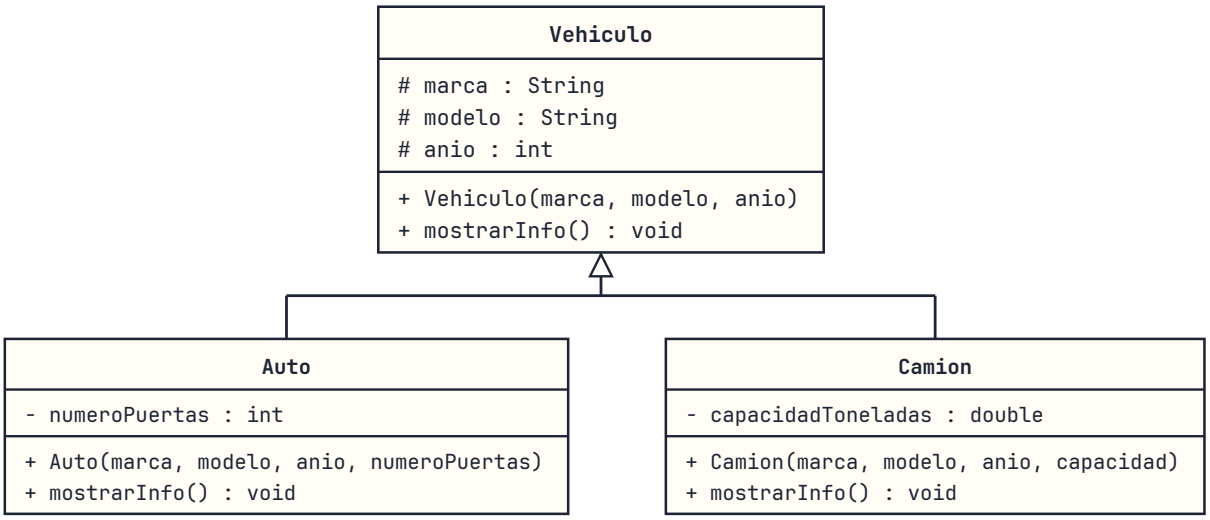
```
Vehículo creado: Italika
Moto lista
El vehículo avanza
La moto Italika acelera
```

El constructor de `Moto` llama primero a `super(marca)` (línea 1) y luego hace lo suyo (línea 2). En `avanzar()`, `super.avanzar()` ejecuta la versión del padre (línea 3) y después la moto agrega la suya (línea 4).

5.3 Los 3 errores

1. El constructor de `Perro` no llama a `super(nombre)`, y `Animal` no tiene constructor vacío: no compila. Corrección: poner `super(nombre);` como primera línea.
2. `nombre` es `private` en `Animal`, así que `Perro` no puede usarlo directamente. Corrección: hacerlo `protected` o agregar un `getNombre()` público en `Animal`.
3. `@Override` en `ladrar()`: `Animal` no tiene ningún método `ladrar()`, así que no hay nada que sobrescribir. Corrección: quitar el `@Override` (es un método nuevo) o sobrescribir `hacerSonido()`.

5.4 Vehículos



Vehiculo.java

```java
public class Vehiculo {
    protected String marca;              // protected: las hijas los pueden usar
    protected String modelo;
    protected int anio;

    public Vehiculo(String marca, String modelo, int anio) {
        this.marca = marca;
        this.modelo = modelo;
        this.anio = anio;
    }

    public void mostrarInfo() {
        System.out.println(marca + " " + modelo + " (" + anio + ")");
    }
}
```

Auto.java

```java
public class Auto extends Vehiculo {
    private int numeroPuertas;

    public Auto(String marca, String modelo, int anio, int numeroPuertas) {
        super(marca, modelo, anio);      // el padre inicializa lo suyo
        this.numeroPuertas = numeroPuertas;
    }

    @Override
    public void mostrarInfo() {
        System.out.print("[Auto] ");
        super.mostrarInfo();             // reutiliza la versión del padre
        System.out.println("  Puertas: " + numeroPuertas);
    }
}
```

Camion.java

```java
public class Camion extends Vehiculo {
    private double capacidadToneladas;

    public Camion(String marca, String modelo, int anio, double capacidadToneladas) {
        super(marca, modelo, anio);
        this.capacidadToneladas = capacidadToneladas;
    }

    @Override
    public void mostrarInfo() {
        System.out.print("[Camión] ");
        super.mostrarInfo();
        System.out.println("  Capacidad: " + capacidadToneladas + " toneladas");
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Auto a = new Auto("Nissan", "Versa", 2022, 4);
        Camion c = new Camion("Volvo", "FH", 2020, 18.5);
        a.mostrarInfo();
        c.mostrarInfo();
    }
}
```

Consola

```
[Auto] Nissan Versa (2022)
  Puertas: 4
[Camión] Volvo FH (2020)
  Capacidad: 18.5 toneladas
```

LECCIÓN 6

6.1 ¿Sobrecarga o sobrescritura?

a) Sobrecarga (misma clase, parámetros distintos). **b)** Sobrescritura. **c)** Sobrecarga (de constructores). **d)** Sobrescritura. **e)** Sobrecarga.

6.2 ¿Qué imprime?

Consola

```
Guau
Miau
...
Miau
```

Polimorfismo: cada objeto ejecuta su propia versión de `hablar()`. `Pollito` no sobrescribió nada, así que usa la versión heredada de `Animal`: «...».

6.3 ¿Compila o no?

a) ✘ `Tambor` no implementa el método abstracto `tocar()`.
b) ✘ No se pueden crear objetos de una clase abstracta.
c) ✔ Una guitarra **es un** instrumento.
d) ✘ Otra vez `new` de una clase abstracta (y además, una variable `Guitarra` no puede guardar un instrumento cualquiera).
e) ✔ Imprime «Rasgueo»: se ejecuta la versión del objeto real.

6.4 Empleados

Empleado.java

```java
public abstract class Empleado {
    protected String nombre;

    public Empleado(String nombre) {
        this.nombre = nombre;
    }

    public abstract double calcularSueldo();      // cada tipo lo calcula distinto

    public void mostrar() {                        // método normal que usa el abstracto
        System.out.println(nombre + " gana $" + calcularSueldo());
    }
}
```

EmpleadoFijo.java

```java
public class EmpleadoFijo extends Empleado {
    private double sueldoMensual;

    public EmpleadoFijo(String nombre, double sueldoMensual) {
        super(nombre);
        this.sueldoMensual = sueldoMensual;
    }

    @Override
    public double calcularSueldo() {
        return sueldoMensual;
    }
}
```

EmpleadoPorHoras.java

```java
public class EmpleadoPorHoras extends Empleado {
    private int horas;
    private double pagoPorHora;

    public EmpleadoPorHoras(String nombre, int horas, double pagoPorHora) {
        super(nombre);
        this.horas = horas;
        this.pagoPorHora = pagoPorHora;
    }

    @Override
    public double calcularSueldo() {
        return horas * pagoPorHora;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Empleado empleados[] = new Empleado[3];
        empleados[0] = new EmpleadoFijo("Ana", 15000);
        empleados[1] = new EmpleadoPorHoras("Luis", 80, 120.5);
        empleados[2] = new EmpleadoFijo("Sofía", 18000);

        for (int i = 0; i < empleados.length; i++) {
            empleados[i].mostrar();              // polimorfismo
        }
    }
}
```

Consola

```
Ana gana $15000.0
Luis gana $9640.0
Sofía gana $18000.0
```

LECCIÓN 7

7.1 ¿Qué imprime?

Consola

```
5
24
8
46
```

`length` es 5. `n[1] + n[3]` = 8 + 16 = 24. `n[0]` pasa a valer 23 − 15 = 8. El `for` va de 2 en 2 (posiciones 0, 2 y 4): 8 + 15 + 23 = 46.

7.2 ¿Qué excepción lanza?

a) `ArrayIndexOutOfBoundsException`: los índices válidos son 0, 1 y 2.
b) `NumberFormatException`: «12a» no es un número.
c) `NullPointerException`: `nombres[0]` vale `null` (nadie le asignó un texto).
d) `ArithmeticException`: división entera entre cero.
e) Ninguna (era la trampa): con `double`, dividir entre 0 no truena; imprime `Infinity`.

7.3 ¿Qué imprime?

Consola

```
[A, X, C] 3
false
```

Después de los tres `add`: `[A, B, C]`. `add(1, "X")` inserta y recorre: `[A, X, B, C]`. `remove(2)` borra la posición 2, que ahora es «B»: `[A, X, C]`. Por eso `contains("B")` da `false`.

7.4 Calificaciones del grupo

Alumno.java

```java
public class Alumno {
    private String nombre;
    private double calificacion;

    public Alumno(String nombre, double calificacion) {
        this.nombre = nombre;
        this.calificacion = calificacion;
    }

    public String getNombre() {
        return nombre;
    }

    public double getCalificacion() {
        return calificacion;
    }

    @Override
    public String toString() {
        return nombre + " (" + calificacion + ")";
    }
}
```

GrupoApp.java

```java
import java.util.ArrayList;
import java.util.InputMismatchException;
import java.util.Scanner;

public class GrupoApp {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);
        ArrayList<Alumno> grupo = new ArrayList<>();
        int opcion;

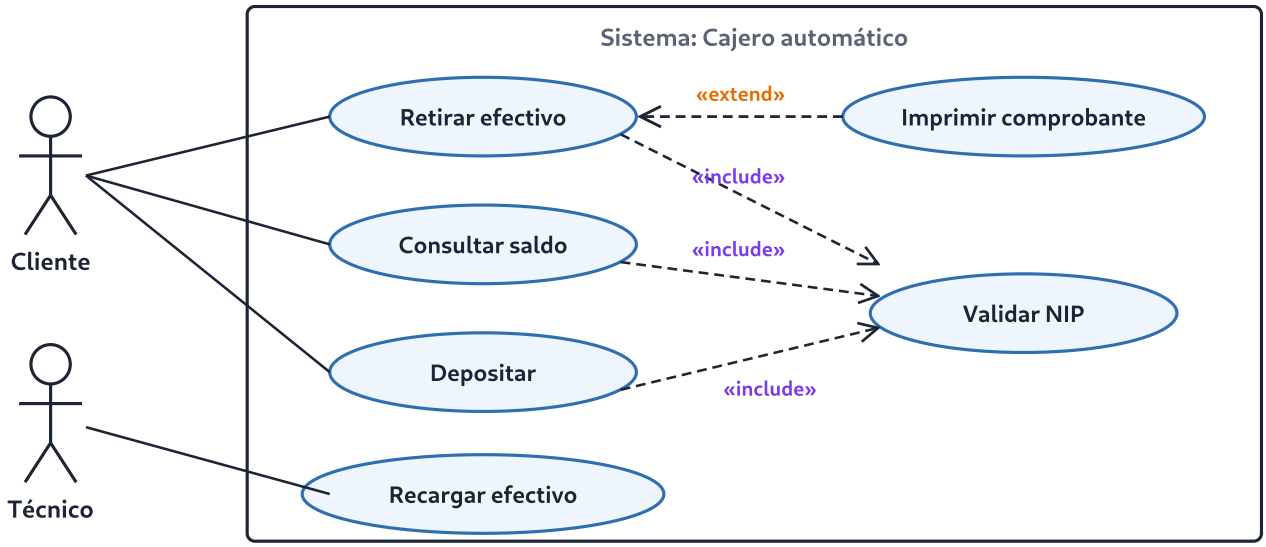
        do {
            System.out.println("1. Agregar  2. Mostrar  3. Buscar  4. Promedio  5. Salir");
            try {
                opcion = sn.nextInt();
            } catch (InputMismatchException e) {
                opcion = 0;                     // cae en el default: "Opción inválida"
            }
            sn.nextLine();                      // limpia el Enter (o lo que escribieron mal)

            switch (opcion) {
                case 1: {
                    System.out.println("Nombre:");
                    String nombre = sn.nextLine();
                    double calificacion = -1;
                    while (calificacion < 0 || calificacion > 100) {
                        System.out.println("Calificación (0 a 100):");
                        try {
                            calificacion = sn.nextDouble();
                        } catch (InputMismatchException e) {
                            System.out.println("Eso no es un número");
                        }
                        sn.nextLine();
                    }
                    grupo.add(new Alumno(nombre, calificacion));
                    System.out.println("Alumno agregado");
                    break;
                }
```

```
                case 2:
                    if (grupo.isEmpty()) {
                        System.out.println("No hay alumnos");
                    } else {
                        for (Alumno a : grupo) {
                            System.out.println(a);           // usa toString()
                        }
                    }
                    break;
                case 3: {
                    System.out.println("Nombre a buscar:");
                    String buscado = sn.nextLine();
                    boolean encontrado = false;
                    for (Alumno a : grupo) {
                        if (a.getNombre().equalsIgnoreCase(buscado)) {
                            System.out.println("Encontrado: " + a);
                            encontrado = true;
                        }
                    }
                    if (!encontrado) {
                        System.out.println("No existe ese alumno");
                    }
                    break;
                }
                case 4:
                    if (grupo.isEmpty()) {
                        System.out.println("No hay alumnos");
                    } else {
                        double suma = 0;
                        for (Alumno a : grupo) {
                            suma = suma + a.getCalificacion();
                        }
                        System.out.println("Promedio del grupo: " + (suma / grupo.size()));
                    }
                    break;
                case 5:
                    System.out.println("Adiós");
                    break;
                default:
                    System.out.println("Opción inválida");
            }
        } while (opcion != 5);
    }
}
```

ANEXO

A.1 Cajero automático



> Los actores van **afuera** del rectángulo. Las tres operaciones **incluyen** «Validar NIP» porque pasa siempre. «Imprimir comprobante» **extiende** a «Retirar efectivo» porque solo pasa si el cliente lo pide.

EXAMEN DE PRÁCTICA

I. Teoría

1. **Clase:** molde que define atributos y métodos (por ejemplo, `Habitacion`). **Objeto:** un elemento concreto creado con `new` a partir de la clase, con sus propios valores (la habitación 101, de $800 por noche).
2. Ocultar los datos de un objeto y permitir usarlos solo mediante sus métodos, para proteger su integridad. Se logra con atributos `private` y getters/setters `public` que validan.
3. **Sobrecarga:** mismo nombre y distintos parámetros, en la misma clase. **Sobrescritura:** una clase hija redefine un método heredado con los mismos parámetros (`@Override`).
4. Una clase declarada `abstract` que representa una idea general; puede tener métodos abstractos (sin cuerpo) que sus hijas deben implementar. **No** se pueden crear objetos de ella.
5. `super(...)` llama al constructor del padre (en la primera línea del constructor de la hija). `super.metodo()` ejecuta la versión del padre de un método sobrescrito.
6. Que pertenece a la clase y no a cada objeto: hay una sola copia, compartida por todos, y se usa con el nombre de la clase (`Clase.miembro`) sin crear objetos.
7. Porque los `String` son objetos: `==` compara si son el mismo objeto (la referencia), mientras que `equals()` compara el contenido.
8. Es un método especial que inicializa al objeto cuando se crea con `new`. Se llama igual que la clase y no tiene tipo de retorno (ni siquiera `void`); un método normal sí lo tiene y se ejecuta cada vez que lo llamas.
9. Que un mismo método se comporte distinto según el objeto real que lo ejecuta. Ejemplo: en un arreglo `Habitacion[]`, `mostrarInfo()` muestra cosas distintas para una sencilla, una doble o una suite.
10. Abstracción, encapsulamiento, herencia y polimorfismo.

II. ¿Qué imprime?

Ejercicio 1:

Consola

```
Creando algo
Creando cuadro
Soy una figura
Soy un cuadro de lado 5
```

Cada `new` ejecuta el constructor de `Figura` (el de `Cuadro` llega ahí por `super("cuadro")`). Luego, por polimorfismo, `f2` usa el `describir()` de `Cuadro`.

Ejercicio 2:

Consola

```
1 3
Vendidos: 3
321
```

`vendidos` es `static` (compartido): cada boleto nuevo lo sube y copia ese valor en su propio `numero` (1, 2 y 3). El último `for` recorre al revés y pega los números como texto: «321».

III. Los 5 errores

1. `public void Libro(...)`: el `void` lo convierte en un método normal, y por eso `new Libro("Java", 300)` no compila. Va sin `void`: `public Libro(String titulo, int paginas)`.
2. `getPaginas()` no tiene `return`: debe ser `return paginas;`.
3. Falta el `;` después de `new Libro("Java", 300)`.
4. `l.getTitulo() == "Java"` compila, pero es un error de lógica: los textos se comparan con `l.getTitulo().equals("Java")`.
5. `l.paginas = 500;`: `paginas` es `private`. Habría que agregar un `setPaginas()` y usarlo.

IV. Del diagrama al código

Persona.java

```java
public abstract class Persona {
    protected String nombre;
    protected int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public void presentarse() {
        System.out.println("Hola, soy " + nombre + " y tengo " + edad + " años");
    }

    public abstract double calcularPago();
}
```

Alumno.java

```java
public class Alumno extends Persona {
    private String matricula;
    private double colegiatura;

    public Alumno(String nombre, int edad, String matricula, double colegiatura) {
        super(nombre, edad);
        this.matricula = matricula;
        this.colegiatura = colegiatura;
    }

    @Override
    public double calcularPago() {
        return colegiatura;
    }
}
```

Profesor.java

```java
public class Profesor extends Persona {
    private int horasClase;
    private double pagoPorHora;

    public Profesor(String nombre, int edad, int horasClase, double pagoPorHora) {
        super(nombre, edad);
        this.horasClase = horasClase;
        this.pagoPorHora = pagoPorHora;
    }

    @Override
    public double calcularPago() {
        return horasClase * pagoPorHora;
    }
}
```

Escuela.java

```java
import java.util.ArrayList;

public class Escuela {
    public static void main(String[] args) {
        ArrayList<Persona> personas = new ArrayList<>();
        personas.add(new Alumno("Ana", 19, "A001", 8500.0));
        personas.add(new Profesor("Carlos", 45, 20, 350.0));

        for (Persona p : personas) {          // polimorfismo con for-each
            p.presentarse();
            System.out.println("Pago: $" + p.calcularPago());
        }
    }
}
```

Consola

```
Hola, soy Ana y tengo 19 años
Pago: $8500.0
Hola, soy Carlos y tengo 45 años
Pago: $7000.0
```

V. Tienda en línea

- **Actores** (afuera del rectángulo): Cliente y Administrador.
- **Casos de uso del cliente:** Buscar productos, Agregar al carrito y Pagar.
- **Casos de uso del administrador:** Dar de alta productos y Consultar ventas.
- **«include»:** de Pagar hacia Validar tarjeta (pasa siempre).
- **«extend»:** de Aplicar cupón hacia Pagar (solo si el cliente quiere).

💡 **¿Y AHORA?**

Lo que hayas fallado, márcalo y repasa esa lección. Si algo sigue sin quedar claro, pregúntamelo en el chat con el número de la práctica y lo vemos con calma. ¡Mucho éxito en tu examen!